L3: Bellman’s Equations & Dynamic Programming

EECE 576N | From MDPs to Deep RL | Fall 2026

Cyrus Neary | cyrus.neary@ubc.ca

Last lecture…

R- Reward function

$R(s, a) \in \mathbb{R}$ — A function defining the reward the agent receives for transitioning from state $s \in S$ to $s' \in S$ under action $a \in A$.

A - Actions

Set of all possible actions the agent could take.

$$\mathscr{M} = (S, A, T, R, \gamma, \mu)$$

μ - Initial state distribution

$\mu(s)$ denotes the probability that the initial state is s.

S - States

Set of all possible states from which the agent will make decisions.

γ - Discount factor

$\gamma \in [0,1]$ — a modeling parameter that reduces the impact of future rewards on decisions made now.

T - Transition function

$s' \sim T(\cdot \mid s, a)$ a stochastic transition function defining the probability of the agent transitioning to next state $s' \in S$, given that it executed action $a \in A$ from state $s \in S$.

Last lecture…

$$V^{\pi}(s) = \mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t r_t \mid \pi, s_0 = s\right]$$

Intuition: How “valuable” is it for the agent to be in state s, if it follows policy π from that point onwards?

$$V^{\pi}(s) = \sum_{a \in A} \pi(a \mid s) Q^{\pi}(s, a) \qquad Q^{\pi}(s, a) = R(s, a) + \gamma \sum_{s' \in S} T(s' \mid s, a) V^{\pi}(s')$$

$$Q^{\pi}(s, a) = \mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t r_t \mid \pi, s_0 = s, a_0 = a\right]$$

Intuition: How “valuable” is it for the agent to take action a from state s, if it follows policy π from that point onwards?

Last lecture…

A policy $\pi^*(\cdot \mid s)$ is considered optimal whenever

$$V^{\pi^*}(s) \geq V^{\pi}(s), \quad \forall s \in S, \forall \pi \in \Pi^{HR}$$

The optimal value of an MDP is defined as:

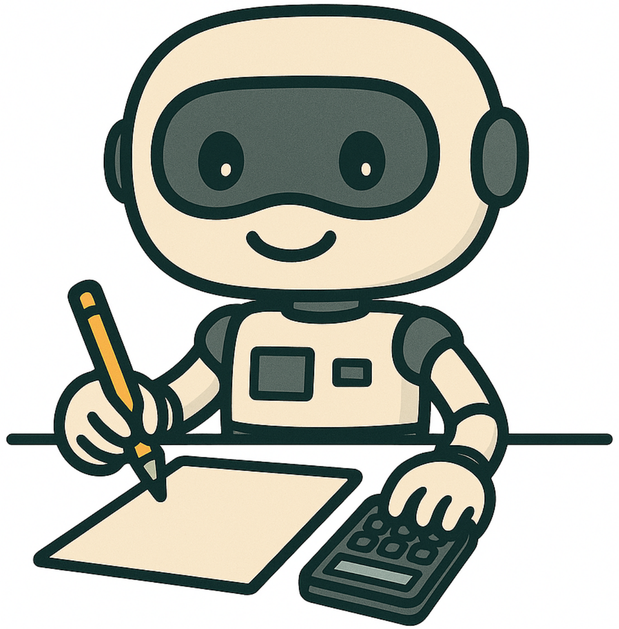
$$V^*(s) := \sup_{\pi \in \Pi^{HR}} V^{\pi}(s)$$

An optimal policy $\pi^* \in \Pi^{\{HR,MR,HD,MD\}}$ exists when:

$$V^{\pi^*}(s) \geq V^*(s), \quad \forall s \in S$$

This lecture…

How can we solve for optimal policies?



This lecture…

How can we solve for optimal policies?

One solution technique: Dynamic programming!

Dynamic programming

41 languages

Article Talk | Read Edit View history Tools

From Wikipedia, the free encyclopedia

Not to be confused with Dynamic programming language or Dynamic problem.

Dynamic programming is both a mathematical optimization method and an algorithmic paradigm. The method was developed by Richard Bellman in the 1950s and has found applications in numerous fields, from aerospace engineering to economics.

In both contexts it refers to simplifying a complicated problem by breaking it down into simpler sub-problems in a recursive manner. While some decision problems cannot be taken apart this way, decisions that span several points in time do often break apart recursively. Likewise, in computer science, if a problem can be solved optimally by breaking it into sub-problems and then recursively finding the optimal solutions to the sub-problems, then it is said to have *optimal substructure*.

If sub-problems can be nested recursively inside larger problems, so that dynamic programming methods are applicable, then there is a relation between the value of the larger problem and the values of the sub-problems.[1] In the optimization literature this relationship is called the Bellman equation.

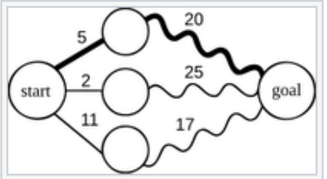
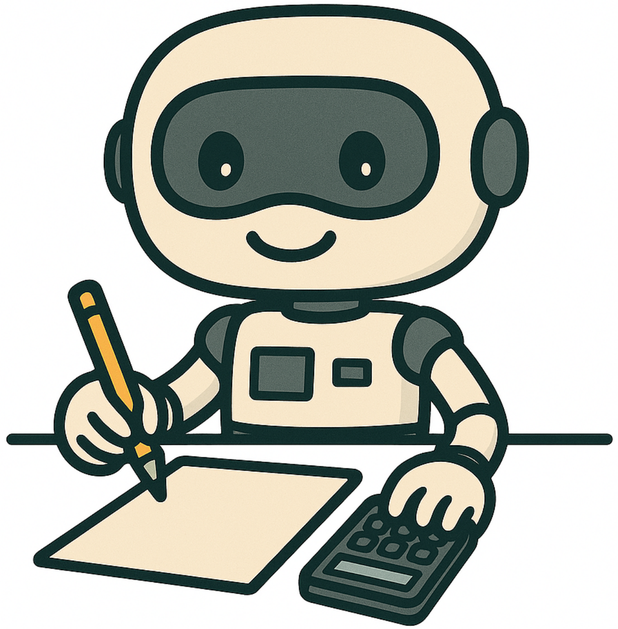
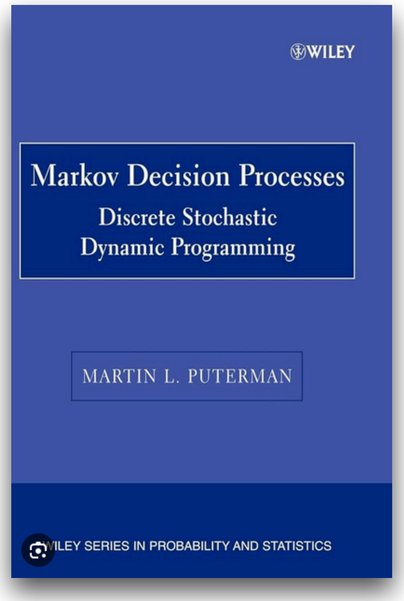


Figure 1. Finding the shortest path in a graph using optimal substructure; a straight line indicates a single edge; a wavy line indicates a shortest path between the two vertices it connects (among other paths, not shown, sharing the same two vertices); the bold line is the overall shortest path from start to goal.

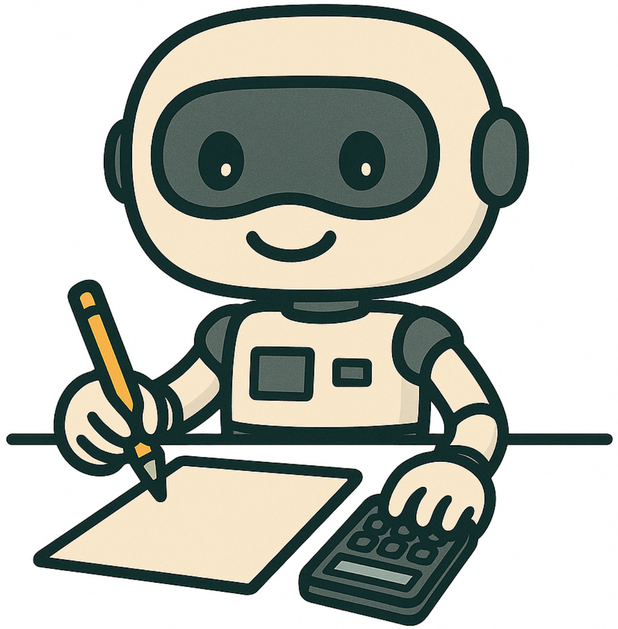


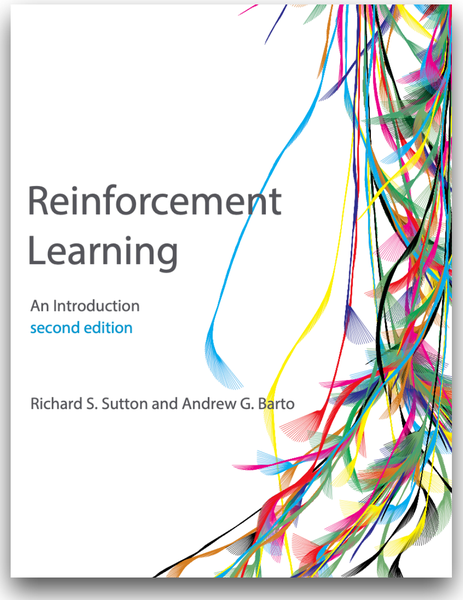
This lecture…

How can we solve for optimal policies?



Chapters 6.1-6.4

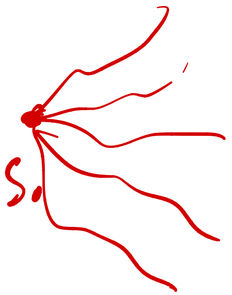




Chapter 4

Why use dynamic programming?

What would an alternate, naieve approach look like? What are the drawbacks?

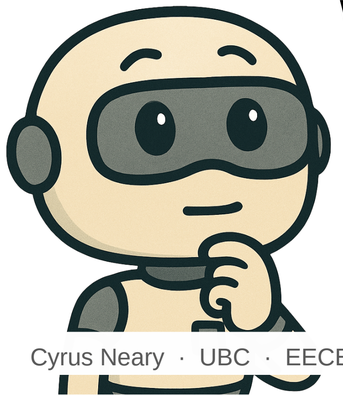


- enumerate all possible trajectories and compute expected returns.
- MDP with branching factor b, and time horizon T, requires $O(b^T)$ computational effort to consider all those futures.

How do we naturally solve long-horizon decision problems?

- Naturally break problems into stages. Think about what a "good" next state is, and try to get there first.
- Make decisions now that put you in a state s that aligns with your long-term goals.

What is the core intuition behind the use of dynamic programming?



Solve small problems and re-use their solutions ~~when~~ as a part of larger problems.

Bellman's equation

$$V^{\pi}(s) = \sum_{a \in A} \pi(a \mid s) \left[R(s,a) + \gamma \sum_{s' \in S} T(s' \mid s, a) V^{\pi}(s') \right]$$



Bellman's equation

$$V^{\pi}(s) = \mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t r_t \mid \pi, s_0 = s\right]$$ definition

$$V^{\pi}(s) = \sum_{a \in A} \pi(a \mid s) \left[R(s,a) + \gamma \sum_{s' \in S} T(s' \mid s, a) V^{\pi}(s') \right]$$

Value at state s



Bellman's equation

Immediate reward from state s under policy π.

$$V^{\pi}(s) = \sum_{a \in A} \pi(a \mid s) \left[R(s,a) + \gamma \sum_{s' \in S} T(s' \mid s, a) V^{\pi}(s') \right]$$

Value at state s



Bellman's equation

Immediate reward from state s under policy π.

$$V^{\pi}(s) = \sum_{a \in A} \pi(a \mid s) \left[R(s,a) + \gamma \sum_{s' \in S} T(s' \mid s, a) V^{\pi}(s') \right]$$

Value at state s

Probability of transitioning from s to new state s' under policy π.



Bellman's equation

Immediate reward from state s under policy π.

$$V^{\pi}(s) = \sum_{a \in A} \pi(a \mid s) \left[R(s,a) + \gamma \sum_{s' \in S} T(s' \mid s, a) V^{\pi}(s') \right]$$

Value at state s

Probability of transitioning from s to new state s' under policy π.

Value of the next state s'



Deriving Bellman's equation

Tower property: $\mathbb{E}[X|Z] = \mathbb{E}[\mathbb{E}[X|Y,Z]|Z]$ (variables X, Y, Z)

$$V^\pi(s) = \mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t r_t \mid \pi, s_0 = s\right]$$

(definition)

$$= \mathbb{E}\left[r_0 + \sum_{t=1}^{\infty} \gamma^t r_t \mid \pi, s_0 = s\right]$$

$$= \mathbb{E}\left[\mathbb{E}\left[r_0 + \sum_{t=1}^{\infty} \gamma^t r_t \mid \pi, s_0 = s, a_0 = a, s_1 = s'\right] \mid \pi, s_0 = s\right]$$

$$\mathbb{E}\left[r_0 + \sum_{t=1}^{\infty} \gamma^t r_t \mid \pi, s_0 = s, a_0 = a, s_1 = s'\right] = R(s,a) + \gamma \mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t r_t \mid \pi, s_0 = s'\right]$$

$\gamma V(s')$

Take outer expectation

$$\mathbb{E}[R(s,a) + \gamma V^\pi(s') \mid \pi, s_0 = s] = \sum_{a,s'} \text{Prob}(a,s')[R(s,a) + \gamma V(s')]$$

$$= \sum_{a \in A} \pi(a|s) \sum_{s' \in S} T(s'|s,a)[R(s,a) + \gamma V^\pi(s)]$$

Bellman's equation

$$V^\pi(s) = \sum_{a \in A} \pi(a \mid s) \left[R(s,a) + \gamma \sum_{s' \in S} T(s' \mid s, a) V^\pi(s')\right]$$

Deriving Bellman's equation

$$V^{\pi}(s) = \mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t r_t \mid \pi, s_0 = s\right]$$

$$\mathbb{E}[R(s,a) + \gamma V^{\pi}(s') \mid \pi, s_0 = s] = \sum_{a,s'} \text{Prob}(a,s')[R(s,a) + \gamma V(s')]$$

$$= \sum_{a \in A} \pi(a|s) \sum_{s' \in S} T(s'|s,a)[R(s,a) + \gamma V^{\pi}(s')]$$

$$= \sum_{a \in A} \pi(a|s) \left\{ \sum_{s' \in S} T(s'|s,a) R(s,a) + \sum_{s' \in S} T(s'|s,a) \gamma V^{\pi}(s') \right\}$$

because $\sum_{s'} T(s') = 1$

$$= \sum_{a \in A} \pi(a|s) \left[R(s,a) + \sum_{s' \in S} T(s' \mid s,a) \gamma V^{\pi}(s') \right]$$

Bellman's equation

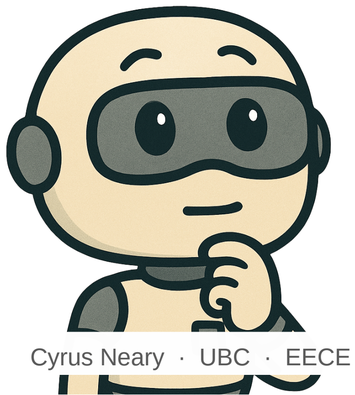
$$V^{\pi}(s) = \sum_{a \in A} \pi(a \mid s) \left[R(s,a) + \gamma \sum_{s' \in S} T(s' \mid s,a) V^{\pi}(s') \right]$$

Why is this recursive structure helpful?

$$V^{\pi}(s) = \mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t r_t \mid \pi, s_0\right]$$

$$V^{\pi}(s) = \sum_{a \in A} \pi(a \mid s) \left[R(s,a) + \gamma \sum_{s' \in S} T(s' \mid s, a) V^{\pi}(s') \right]$$

- Reveals mathematically, the substructure of our intuition on the value function, and provides a path to value function computation.
- Reduces a quantity over infinite sums of paths into a quantity in terms of the value now, and the value at the next step.



Q-functions have an identical recursive structure

$$Q^\pi(s,a) = \mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t r_t \mid \pi, s_0 = s, a_0 = a\right] = \mathbb{E}\left[r_0 + \sum_{t=1}^{\infty} \gamma^t r_t \mid \pi, s_0 = s, a_0 = a\right]$$

again, apply tower property

$$\mathbb{E}\left[\mathbb{E}\left[r_0 + \sum_{t=1}^{\infty} \gamma^t r_t \mid \pi, s_0 = s, a_0 = a, s_1 = s', a_1 = a'\right] \mid \pi, s_0 = s, a_0 = a\right]$$

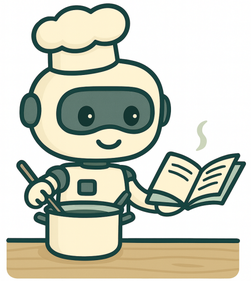
$$R(s,a) + \gamma \mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t r_t \mid \pi, s_0 = s', a_0 = a'\right] = R(s,a) + \gamma Q^\pi(s',a')$$

plugging this into the outer expectation, and writing out the expectation as a sum, we get:

$$Q^\pi(s,a) = R(s,a) + \gamma \sum_{s' \in S} T(s' \mid s,a) \sum_{a' \in A} \pi(a' \mid s') Q^\pi(s',a')$$

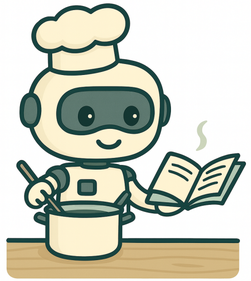
Policy evaluation, policy improvement, policy iteration

The overall recipe:



Policy evaluation, policy improvement, policy iteration

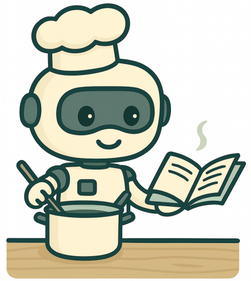
The overall recipe:



Policy evaluation: Given a policy π, compute the value function $V^{\pi}(s)$ of that policy.

Policy evaluation, policy improvement, policy iteration

The overall recipe:

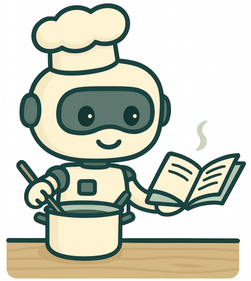


Policy evaluation: Given a policy π, compute the value function $V^{\pi}(s)$ of that policy.

Policy improvement: Use the computed value $V^{\pi}(s)$ function to compute an *improved policy* π' such that $V^{\pi'}(s) \geq V^{\pi}(s), \quad \forall s \in S.$

Policy evaluation, policy improvement, policy iteration

The overall recipe:



Policy evaluation: Given a policy π, compute the value function $V^{\pi}(s)$ of that policy.

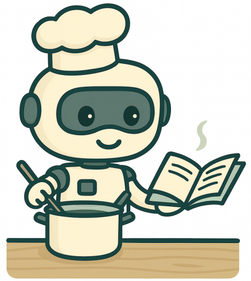
$$V^{\pi} = \sum_{a \in A} \pi(a|s) \left[R(s,a) + \gamma \sum_{s' \in S} T(s'|s,a) V^{\pi}(s') \right]$$

Policy improvement: Use the computed value $V^{\pi}(s)$ function to compute an *improved policy* π' such that $V^{\pi'}(s) \geq V^{\pi}(s), \quad \forall s \in S.$

Policy iteration: Iterate between policy evaluation and policy improvement to solve for an optimal policy π^* and optimal value function $V^*(s)$.

Policy evaluation, policy improvement, policy iteration

The overall recipe:



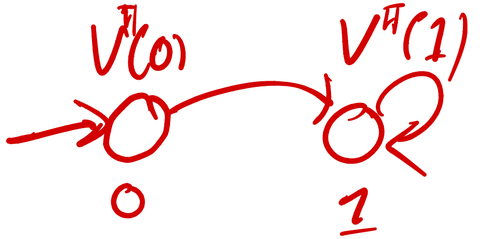
Policy evaluation: Given a policy π, compute the value function $V^{\pi}(s)$ of that policy.

Policy improvement: Use the computed value $V^{\pi}(s)$ function to compute an *improved policy* π' such that $V^{\pi'}(s) \geq V^{\pi}(s), \quad \forall s \in S.$

Policy iteration: Iterate between policy evaluation and policy improvement to solve for an optimal policy π^* and optimal value function $V^*(s)$.

Policy evaluation, linear algebra solution

Suppose we are given a policy π and an MDP model $\mathscr{M}$.
How can we compute the policy's value?



$$V^\pi(s) = \sum_{a \in A} \pi(a \mid s) \left[R(s,a) + \gamma \sum_{s' \in S} T(s' \mid s, a) V^\pi(s') \right]$$

$$V_\pi = \begin{bmatrix} V^\pi(0) \\ V^\pi(1) \end{bmatrix}$$

expand $V^\pi(s) = \sum_{a \in A} \pi(a|s) R(s,a) + \gamma \sum_{a \in A} \pi(a|s) \sum_{s' \in S} T(s'|s,a) V^\pi(s')$

Define $R^\pi(s)$

Define $P^\pi(s,s') = \sum_{a \in A} \pi(a|s) T(s'|s,a)$

Then we have $V^\pi(s) = R^\pi(s) + \gamma \sum_{s' \in S} P^\pi(s,s') V^\pi(s') \quad \forall s \in S$

Define

- $V_\pi \in \mathbb{R}^{|S|}$ vector with entries $V^\pi(s)$
- $R_\pi \in \mathbb{R}^{|S|}$ vector with entries $R^\pi(s)$
- $P_\pi \in \mathbb{R}^{|S| \times |S|}$ matrix with entries $P^\pi(s,s')$

$$\Rightarrow V_\pi = R_\pi + \gamma P_\pi V_\pi$$

$$\Rightarrow \boxed{V_\pi = (I - \gamma P_\pi)^{-1} R_\pi}$$

Policy evaluation, linear algebra solution

Suppose we are given a policy π and an MDP model $\mathscr{M}$.
How can we compute the policy's value?

$$V^{\pi}(s) = \sum_{a \in A} \pi(a \mid s) \left[R(s,a) + \gamma \sum_{s' \in S} T(s' \mid s, a) V^{\pi}(s') \right]$$

$$\sum_{s' \in S} P^{\pi}(s,s') V^{\pi}(s')$$

$$\begin{bmatrix} P_{00} & P_{01} & \cdots & P_{0|S|} \\ P_{10} & & & \\ \vdots & & \ddots & \\ P_{|S|0} & & & P_{|S||S|} \end{bmatrix} \begin{bmatrix} V_0 \\ V_1 \\ \vdots \\ V_{|S|} \end{bmatrix}$$

$$P^{\pi} V^{\pi}$$

Policy evaluation, iterative solution

Suppose we are given a policy π and an MDP model $\mathscr{M}$.
How can we compute the policy's value?

$$V^\pi(s) = \sum_{a \in A} \pi(a \mid s) \left[R(s,a) + \gamma \sum_{s' \in S} T(s' \mid s, a) V^\pi(s') \right]$$

Define the Bellman operator $T_\pi : \mathbb{R}^{|S|} \to \mathbb{R}^{|S|}$ as a mapping from value functions to value functions

$$(T_\pi V)(s) = \sum_{a \in A} \pi(a|s) \left[R(s,a) + \gamma \sum_{s' \in S} T(s'|s,a) V(s') \right]$$

Compute $V^\pi(s)$ using the following iterative procedure:

- Pick some initial guess for $V^\pi(s)$. e.g., $V_0(s) = 0, \forall s \in S$.
- For k = 0, 1, 2, …
  - $V_{k+1} \leftarrow T_\pi V_k$

The value function V^π in question is the fixed point $V^\pi = T_\pi V^\pi$.

Policy evaluation, iterative solution

Compute $V^{\pi}(s)$ using the following iterative procedure:

- Pick some initial guess for $V^{\pi}(s)$. e.g., $V_0(s) = 0, \forall s \in S$.
- For k = 0, 1, 2, …
  - $V_{k+1} \leftarrow T_{\pi} V_k$

Policy evaluation, iterative solution

Compute $V^{\pi}(s)$ using the following iterative procedure:

- Pick some initial guess for $V^{\pi}(s)$. e.g., $V_0(s) = 0, \forall s \in S$.
- For k = 0, 1, 2, …
  - $V_{k+1} \leftarrow T_{\pi} V_k$

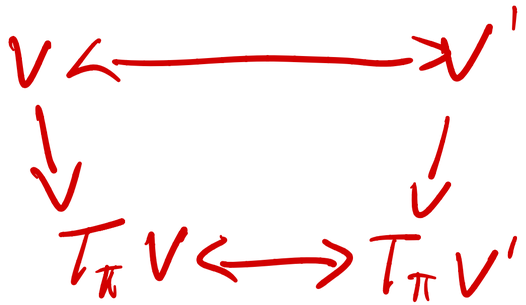
The value function V^{π} in question is the fixed point $V^{\pi} = T_{\pi} V^{\pi}$.

Policy evaluation, iterative solution

Compute $V^\pi(s)$ using the following iterative procedure:

- Pick some initial guess for $V^\pi(s)$. e.g., $V_0(s) = 0, \forall s \in S$.
- For k = 0, 1, 2, …
  - $V_{k+1} \leftarrow T_\pi V_k$

The value function V^π in question is the fixed point $V^\pi = T_\pi V^\pi$.



Proof sketch

Equip $\mathbb{R}^{|S|}$ with the norm $||V||_\infty = \max_{s \in S} |V(s)|$.

Lemma: for any V, V', $||T_\pi V - T_\pi V'||_\infty \leq \gamma ||V - V'||_\infty$.

- Proof of lemma: Homework!!

Consequence of lemma:

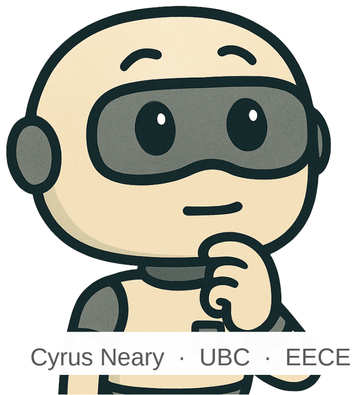
- By the Banach fixed-point theorem, T_π has a **unique fixed point.**
- By definition of T_π and V_π, this unique fixed point must be V_π
- The iterates converge from any starting point V_0.
- The error in the iterates shrinks geometrically with γ:
  - $||T_\pi V_0 - T_\pi V^\pi||_\infty \leq \gamma ||V_0 - V^\pi||_\infty$

When might we use a direct solution to the system of equations vs. using an iterative approach to policy evaluation?

$$V_\pi = (I - \gamma P_\pi)^{-1} R_\pi$$

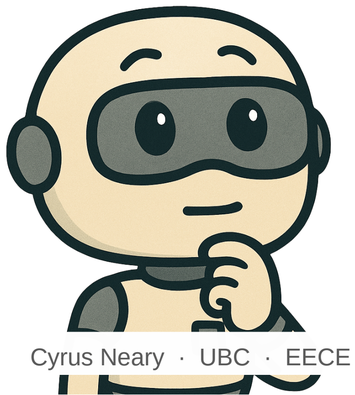
For k = 0, 1, 2, …
$$V_{k+1} \leftarrow T_\pi V_k$$

- For linear algebra solution, we need to invert a matrix of dimension $\mathbb{R}^{|S|} \times \mathbb{R}^{|S|}$. $O(|S|^3)$.
- For iterative approach, we need $O(|S|^2)$ per iterate.
  - ↳ The iterative approach leads to ideas on approximate/locally correct value estimates.



Dynamic programming is efficient for solving MDPs because:

A: It avoids stochasticity by assuming deterministic transitions.
B: It reduces the number of states in the MDP.
C: It re-uses solutions to overlapping subproblems instead of recomputing them.
D: It requires only a single sweep over the state space to converge.
E: I don’t know what’s going on.



Iterative policy evaluation $V_{k+1} \leftarrow T_\pi V_k$ converges to V^π from any starting guess V_0. What property of T_π guarantees this?

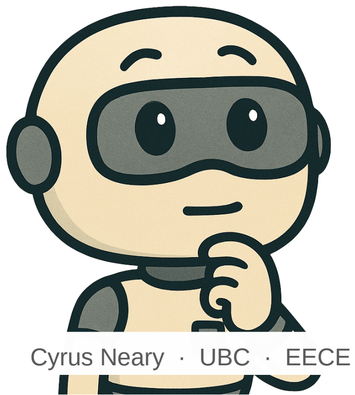
A: T_π is a γ-contraction in the max-norm, so by the Banach fixed-point theorem it has a unique fixed point the iterates converge to.

B: T_π is a linear operator.

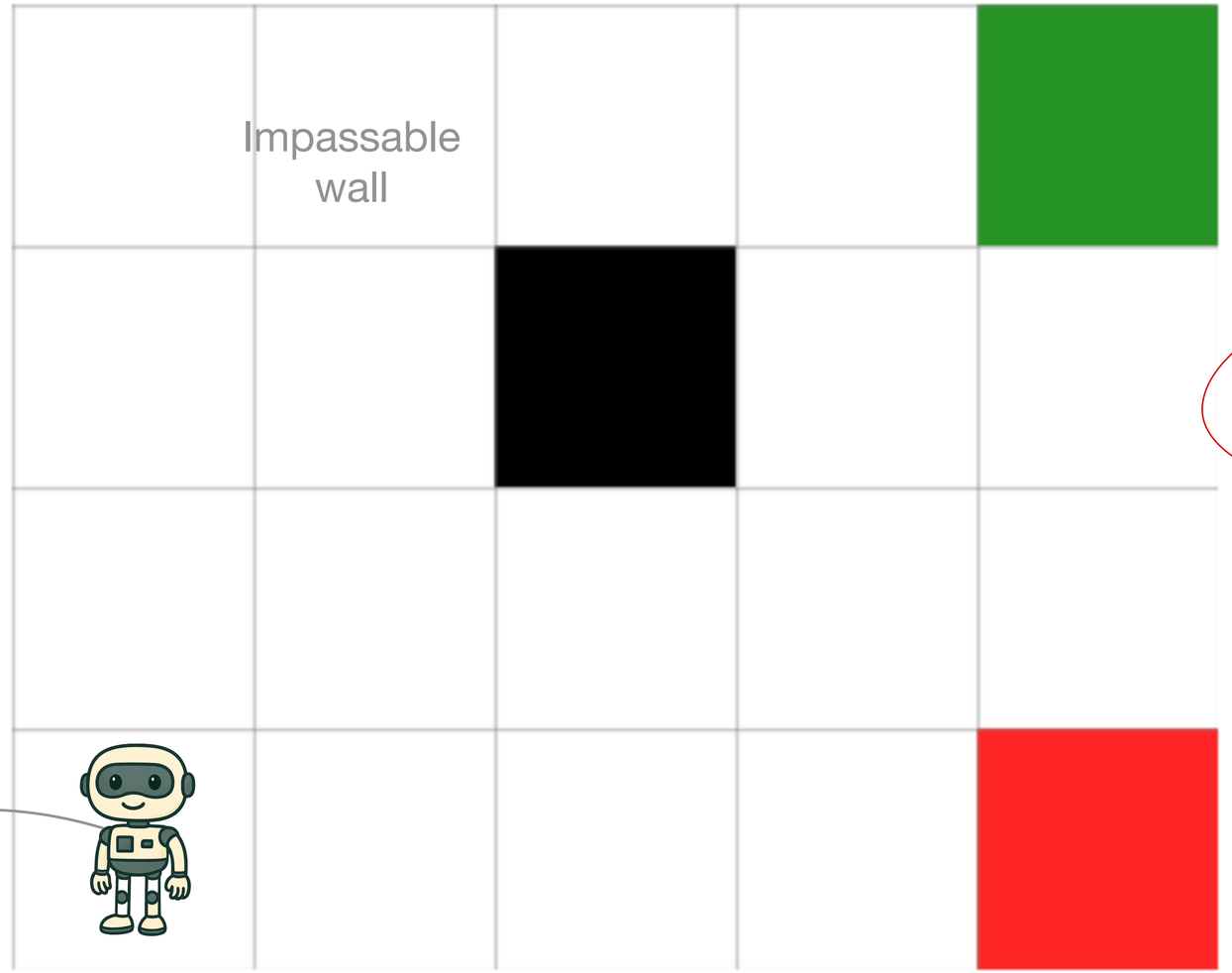
C: T_π maps every value function to the zero function.

D: V_0 must be initialized to V^π for convergence to occur.

E: I don't know what's going on.



Introducing a running gridworld example



Goal state
+10 reward
absorbing

Impassable
wall

Every other step:
-0.1 reward

Dynamics:
“Slip probability” of 0.3

Initial state

Sink state
-5 reward
absorbing

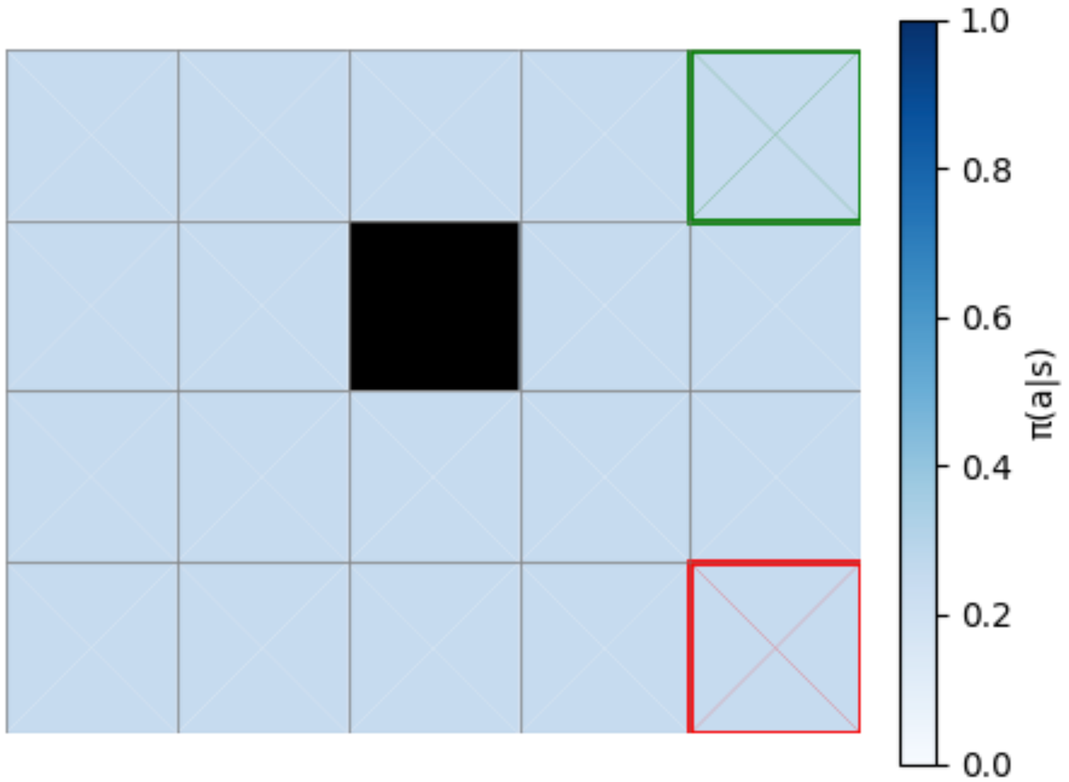
Visualizing two example policies

Uniform distribution over actions

Always try moving right

Visualizing two example policies

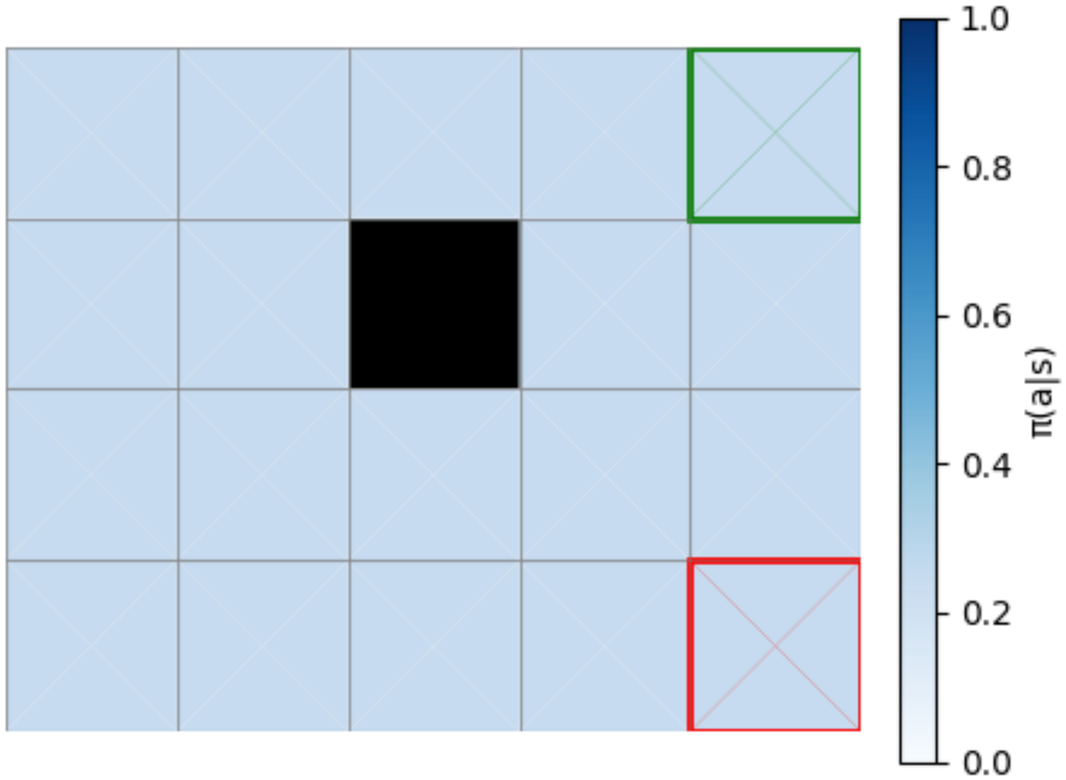
Uniform distribution over actions



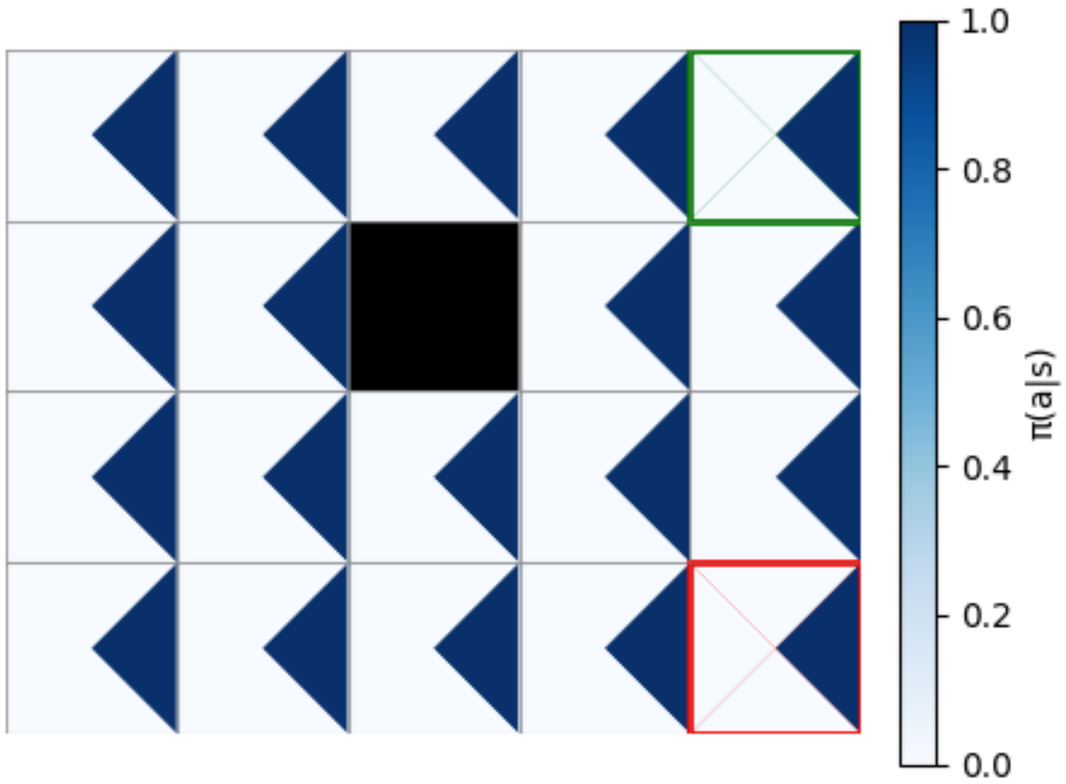
Always try moving right

Visualizing two example policies

Uniform distribution over actions



Always try moving right



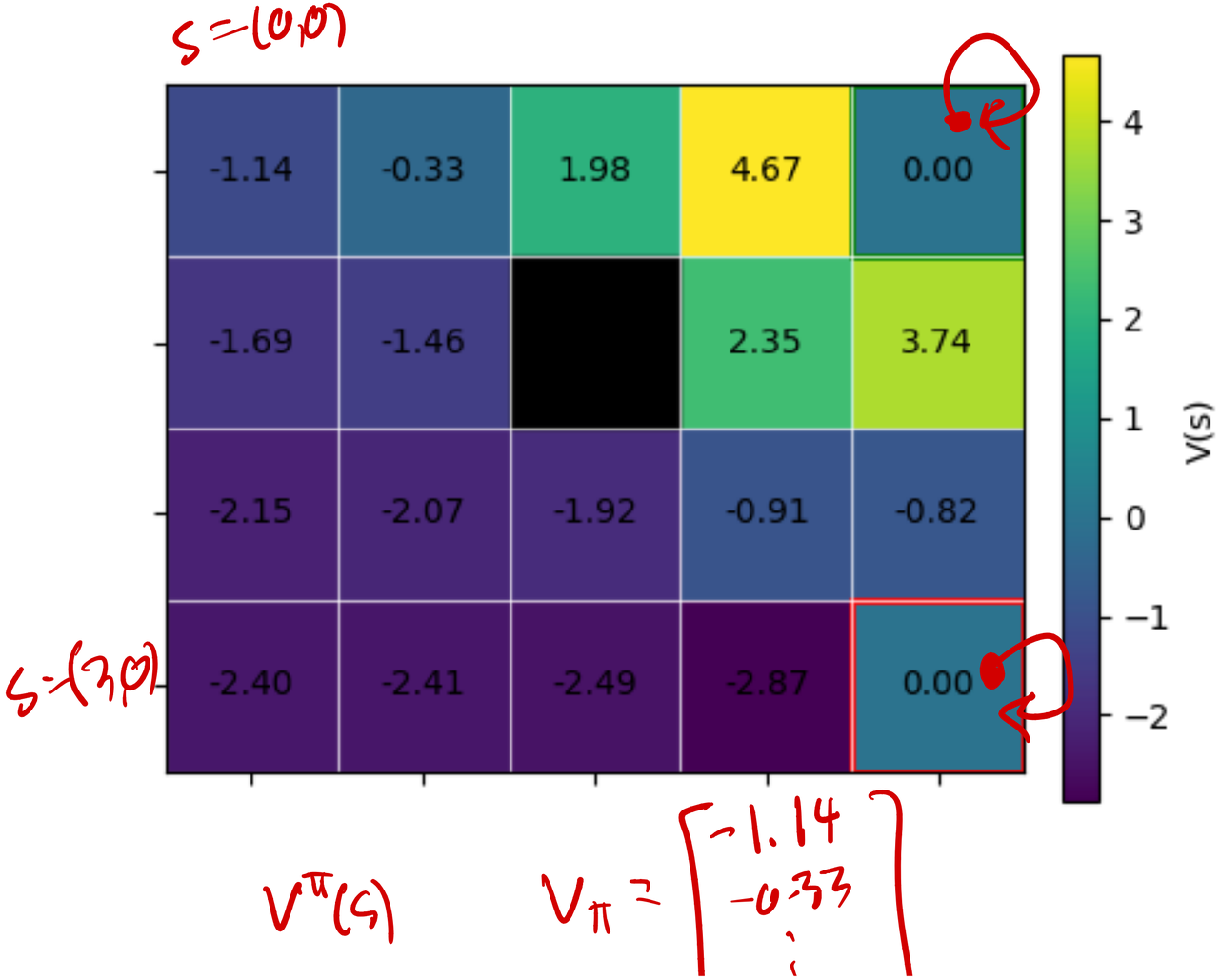
Policy evaluation: visualizing state value function $V^{\pi}(s)$

π_1 Uniform distribution over actions

π_2 Always try moving right

Policy evaluation: visualizing state value function $V^{\pi}(s)$

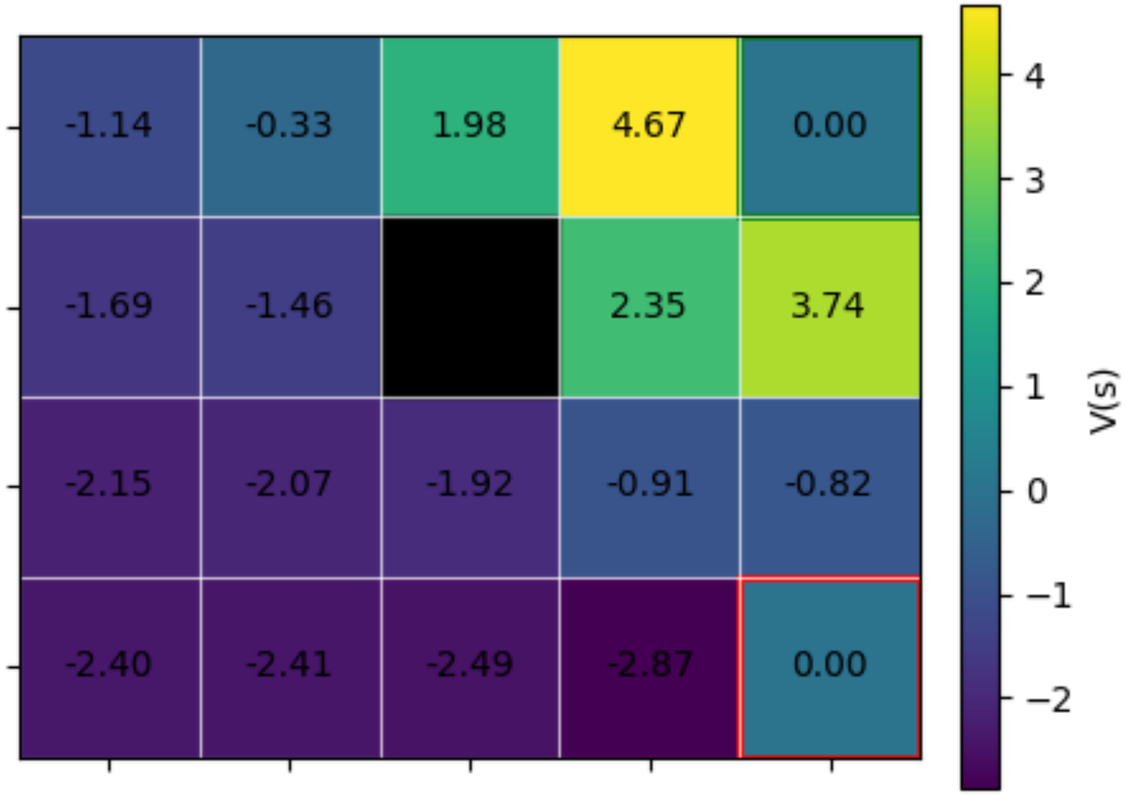
Uniform distribution over actions



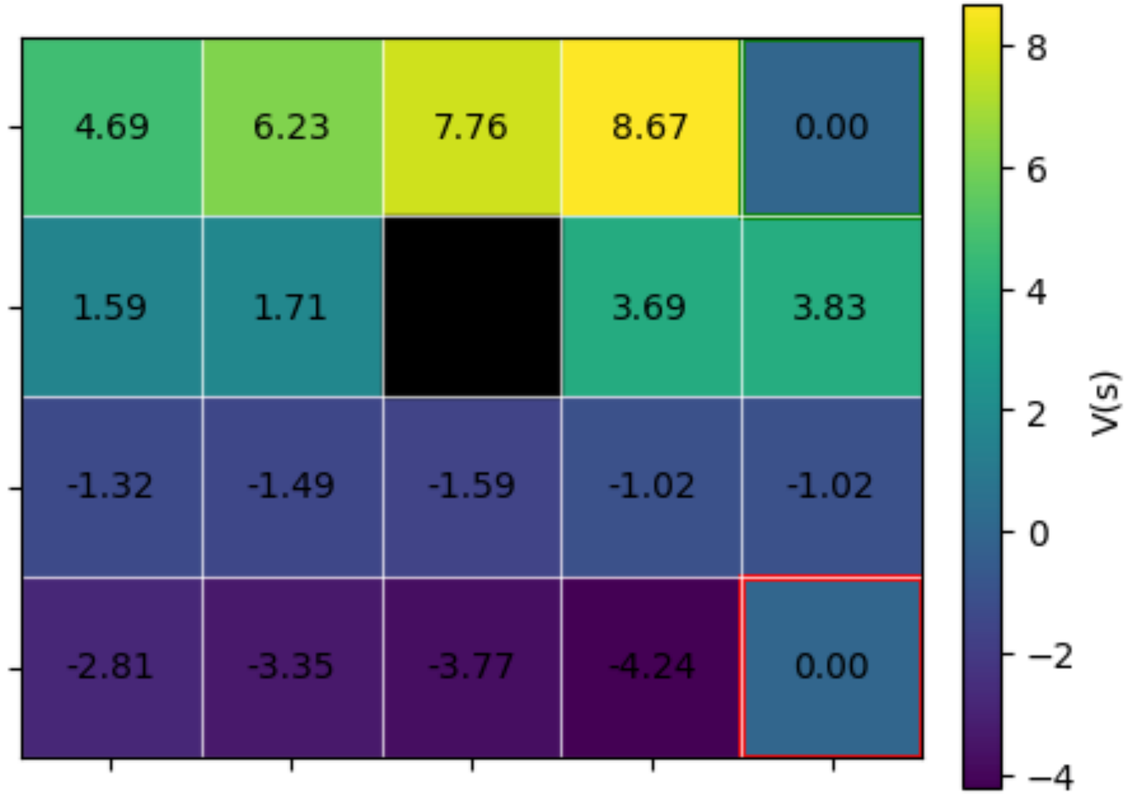
Always try moving right

Policy evaluation: visualizing state value function $V^{\pi}(s)$

Uniform distribution over actions

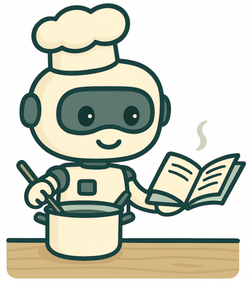


Always try moving right



Policy evaluation, policy improvement, policy iteration

The overall recipe:



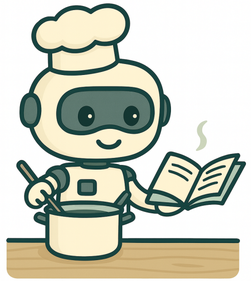
Policy evaluation: Given a policy π, compute the value function $V^{\pi}(s)$ of that policy.

Policy improvement: Use the computed value $V^{\pi}(s)$ function to compute an *improved policy* π' such that $V^{\pi'}(s) \geq V^{\pi}(s), \quad \forall s \in S.$

Policy iteration: Iterate between policy evaluation and policy improvement to solve for an optimal policy π^* and optimal value function $V^*(s)$.

Policy evaluation, policy improvement, policy iteration

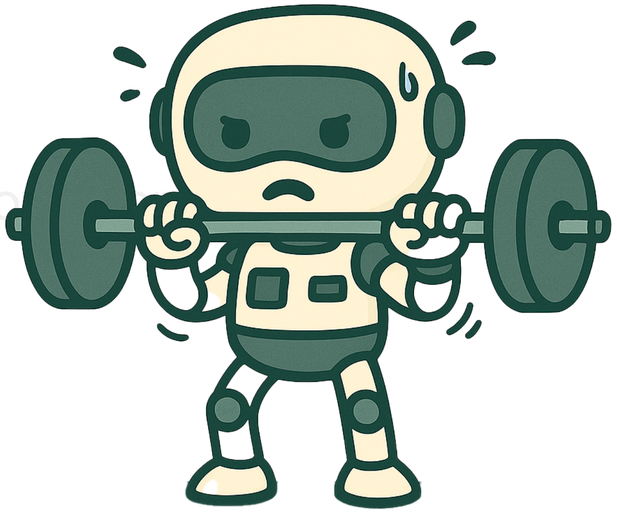
The overall recipe:



Policy evaluation: Given a policy π, compute the value function $V^{\pi}(s)$ of that policy.

Policy improvement: Use the computed value $V^{\pi}(s)$ function to compute an *improved policy* π' such that $V^{\pi'}(s) \geq V^{\pi}(s), \quad \forall s \in S.$

Policy iteration: Iterate between policy evaluation and policy improvement to solve for an optimal policy π^* and optimal value function $V^*(s)$.



Policy Improvement

Consider a given policy π, and its corresponding value $V^\pi(s)$ and state-action value $Q^\pi(s,a)$ functions.

Recall $V^\pi(s) = \sum_{a \in A} \pi(a|s) Q^\pi(s,a)$ and $Q^\pi(s,a) = R(s,a) + \gamma \sum_{s' \in S} T(s'|s,a) V^\pi(s')$

Policy improvement theorem:

Let π, π' be two deterministic policies such that, for all $s \in S$, $Q^\pi(s, \pi'(s)) \geq V^\pi(s)$.

Then $V^{\pi'}(s) \geq V^\pi(s)$ for all states $s \in S$.

Policy Improvement

Consider a given policy π, and its corresponding value $V^{\pi}(s)$ and state-action value $Q^{\pi}(s, a)$ functions.

Policy improvement theorem:

Let π, π' be two deterministic policies such that, for all $s \in S$, $Q^{\pi}(s, \pi'(s)) \geq V^{\pi}(s)$.

Then $V^{\pi'}(s) \geq V^{\pi}(s)$ for all states $s \in S$.

Policy improvement theorem (in words):

If, for every state, the value of counterfactually replacing the action output by $\pi(s)$ with the action output by $\pi'(s)$ before following π thereafter increases the value…

Then policy π' is a strict improvement (in terms of value) over policy π.

Policy Improvement

Policy improvement theorem:

Let π, π' be two deterministic policies such that, for all $s \in S$, $Q^{\pi}(s, \pi'(s)) \geq V^{\pi}(s)$.

Then $V^{\pi'}(s) \geq V^{\pi}(s)$ for all states $s \in S$.

Policy Improvement

Policy improvement theorem:

Let π, π' be two deterministic policies such that, for all $s \in S$, $Q^{\pi}(s, \pi'(s)) \geq V^{\pi}(s)$.

Then $V^{\pi'}(s) \geq V^{\pi}(s)$ for all states $s \in S$.

Proof:

$$
\begin{aligned}
v_\pi(s) &\leq q_\pi(s, \pi'(s)) \\
&= \mathbb{E}[R_{t+1} + \gamma v_\pi(S_{t+1}) \mid S_t = s, A_t = \pi'(s)] && \text{(by (4.6))} \\
&= \mathbb{E}_{\pi'}[R_{t+1} + \gamma v_\pi(S_{t+1}) \mid S_t = s] \\
&\leq \mathbb{E}_{\pi'}[R_{t+1} + \gamma q_\pi(S_{t+1}, \pi'(S_{t+1})) \mid S_t = s] && \text{(by (4.7))} \\
&= \mathbb{E}_{\pi'}[R_{t+1} + \gamma \mathbb{E}_{\pi'}[R_{t+2} + \gamma v_\pi(S_{t+2}) | S_{t+1}, A_{t+1} = \pi'(S_{t+1})] \mid S_t = s] \\
&= \mathbb{E}_{\pi'}\left[R_{t+1} + \gamma R_{t+2} + \gamma^2 v_\pi(S_{t+2}) \mid S_t = s\right] \\
&\leq \mathbb{E}_{\pi'}\left[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \gamma^3 v_\pi(S_{t+3}) \mid S_t = s\right] \\
&\vdots \\
&\leq \mathbb{E}_{\pi'}\left[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \gamma^3 R_{t+4} + \cdots \mid S_t = s\right] \\
&= v_{\pi'}(s).
\end{aligned}
$$

Policy Improvement

Policy improvement theorem:

Let π, π' be two deterministic policies such that, for all $s \in S$, $Q^{\pi}(s, \pi'(s)) \geq V^{\pi}(s)$.

Then $V^{\pi'}(s) \geq V^{\pi}(s)$ for all states $s \in S$.

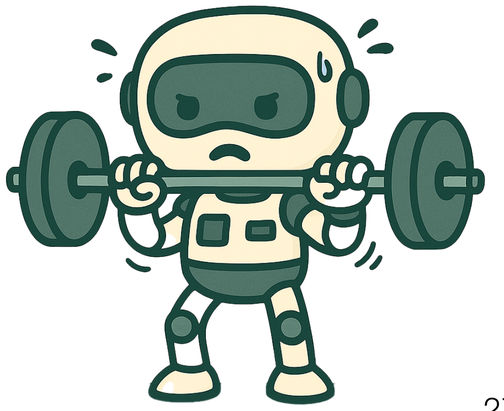
How can we use the policy improvement theorem?

Given the initial policy π and its value functions V^{π}, Q^{π}, construct π' as:

$$\pi'(s) \in \text{argmax}_{a \in A} Q^{\pi}(s, a) = \text{argmax}_{a \in A} \left[R(s, a) + \gamma \sum_{s' \in S} T(s' \mid s, a) V^{\pi}(s') \right]$$

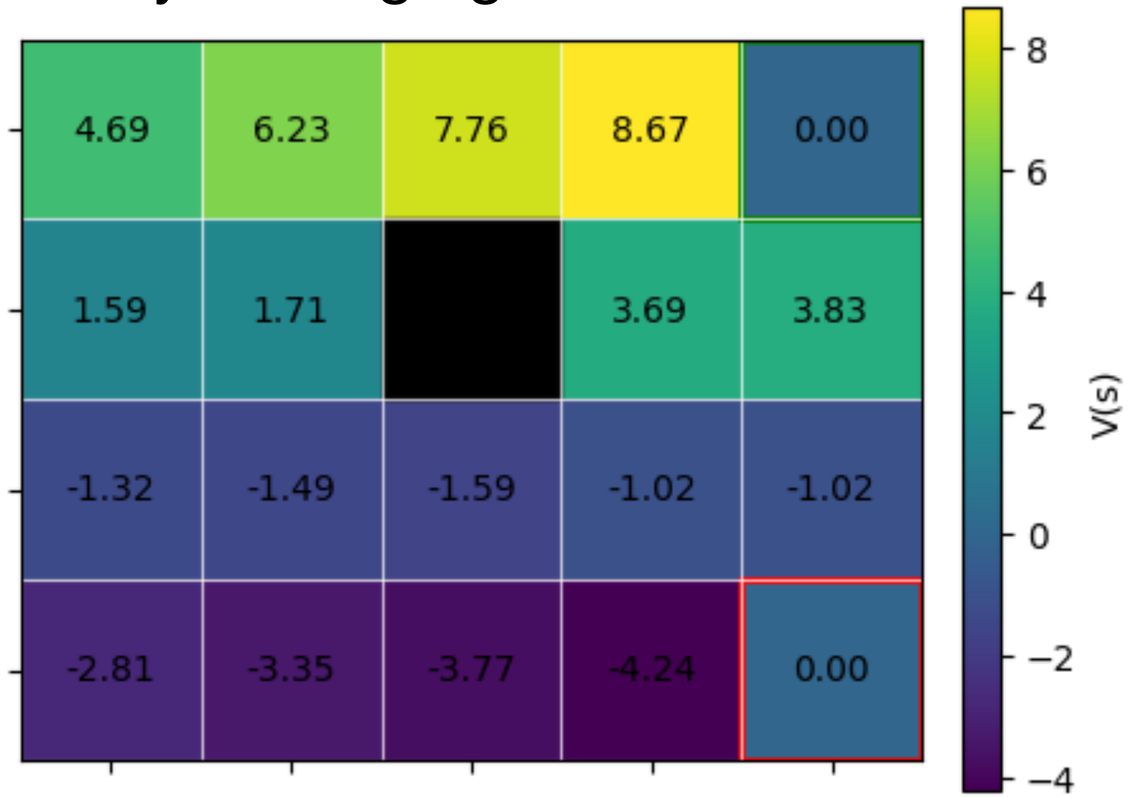
Then, trivially, we have $Q^{\pi}(s, \pi'(s)) \geq V^{\pi}(s)$ and so by the policy improvement theorem $V^{\pi'}(s) \geq V^{\pi}(s)$.

$Q^{\pi}(s, \pi'(s)) \geq Q^{\pi}(s, \pi(s)) = V^{\pi}(s)$



Return to example: Example policy improvement step

π Value for policy: always try moving right



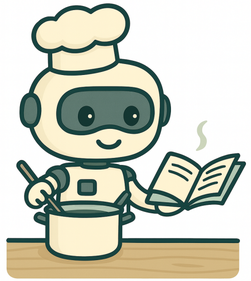
π' After one iteration of policy improvement



V^{π}, $Q^{\pi}(s,a) = R(s,a) + \gamma \sum_{s' \in S} T(s'|s,a) V^{\pi}(s')$

Policy evaluation, policy improvement, policy iteration

The overall recipe:



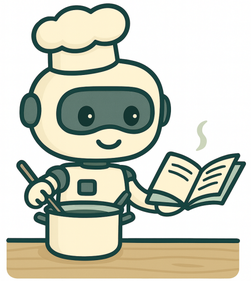
Policy evaluation: Given a policy π, compute the value function $V^{\pi}(s)$ of that policy.

Policy improvement: Use the computed value $V^{\pi}(s)$ function to compute an *improved policy* π' such that $V^{\pi'}(s) \geq V^{\pi}(s), \quad \forall s \in S.$

Policy iteration: Iterate between policy evaluation and policy improvement to solve for an optimal policy π^* and optimal value function $V^*(s)$.

Policy evaluation, policy improvement, policy iteration

The overall recipe:



Policy evaluation: Given a policy π, compute the value function $V^{\pi}(s)$ of that policy.

Policy improvement: Use the computed value $V^{\pi}(s)$ function to compute an *improved policy* π' such that $V^{\pi'}(s) \geq V^{\pi}(s), \quad \forall s \in S.$

Policy iteration: Iterate between policy evaluation and policy improvement to solve for an optimal policy π^* and optimal value function $V^*(s)$.

Policy iteration

Iterate between policy evaluation and policy improvement to solve for an optimal policy π^* and optimal value function $V^*(s)$.

Policy Iteration (using iterative policy evaluation) for estimating $\pi \approx \pi_*$

1. Initialization
   $V(s) \in \mathbb{R}$ and $\pi(s) \in \mathcal{A}(s)$ arbitrarily for all $s \in \mathcal{S}$

2. Policy Evaluation
   Loop:
      $\Delta \leftarrow 0$
      Loop for each $s \in \mathcal{S}$:
         $v \leftarrow V(s)$
         $V(s) \leftarrow \sum_{s',r} p(s', r \mid s, \pi(s)) \left[r + \gamma V(s') \right]$
         $\Delta \leftarrow \max(\Delta, |v - V(s)|)$
   until $\Delta < \theta$ (a small positive number determining the accuracy of estimation)

3. Policy Improvement
   policy-stable $\leftarrow$ *true*
   For each $s \in \mathcal{S}$:
      old-action $\leftarrow \pi(s)$
      $\pi(s) \leftarrow \arg\max_a \sum_{s',r} p(s', r \mid s, a) \left[r + \gamma V(s') \right]$
      If *old-action* $\neq \pi(s)$, then *policy-stable* $\leftarrow$ *false*
   If *policy-stable*, then stop and return $V \approx v_*$ and $\pi \approx \pi_*$; else go to 2

Iterative approach to computing $V^\pi(s)$

Figure courtesy of Sutton & Barto.

Policy iteration convergence

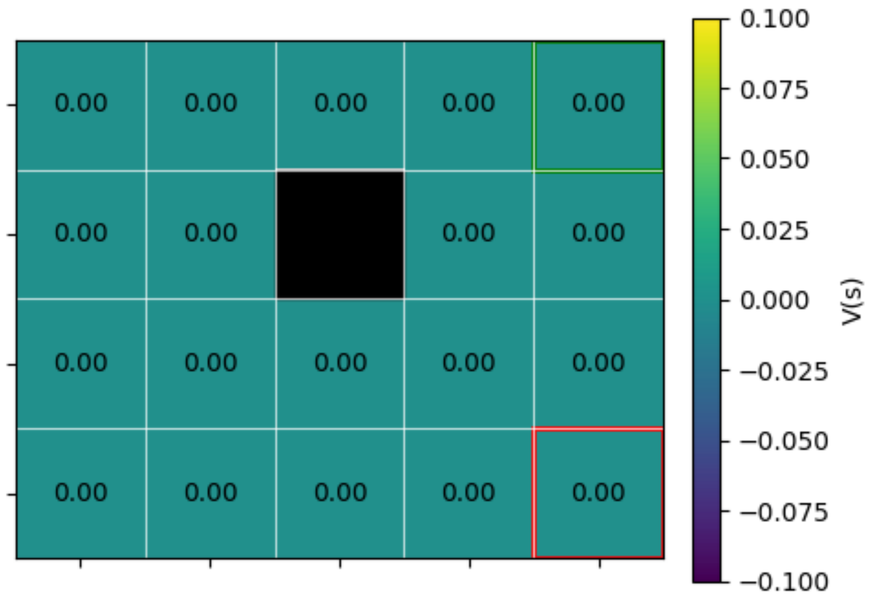
Iterate between policy evaluation and policy improvement to solve for an optimal policy π^* and optimal value function $V^*(s)$.

$$\pi_0 \xrightarrow{\text{E}} v_{\pi_0} \xrightarrow{\text{I}} \pi_1 \xrightarrow{\text{E}} v_{\pi_1} \xrightarrow{\text{I}} \pi_2 \xrightarrow{\text{E}} \cdots \xrightarrow{\text{I}} \pi_* \xrightarrow{\text{E}} v_*,$$

Policy improvement converges in a finite number of steps to an optimal policy π^* and its value function $V^*(s)$, because each improvement step yields a strictly better policy across all states, and the number of distinct deterministic markovian policies is finite.

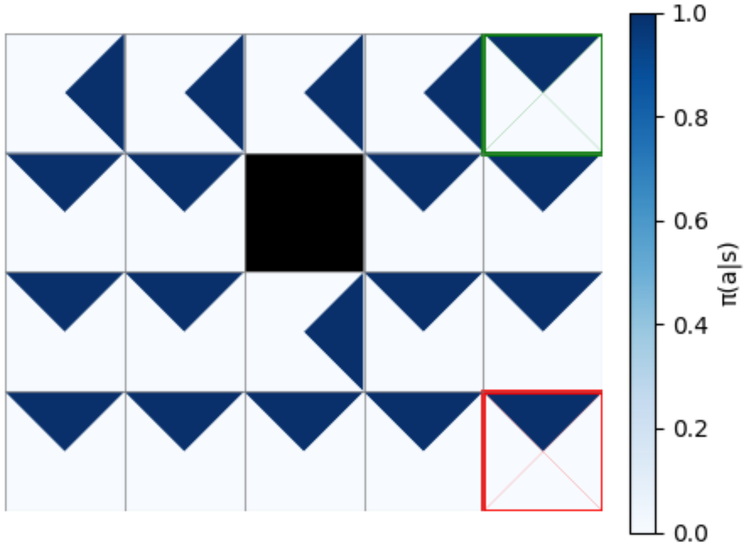
Policy iteration gridworld example

$V_0(s)$

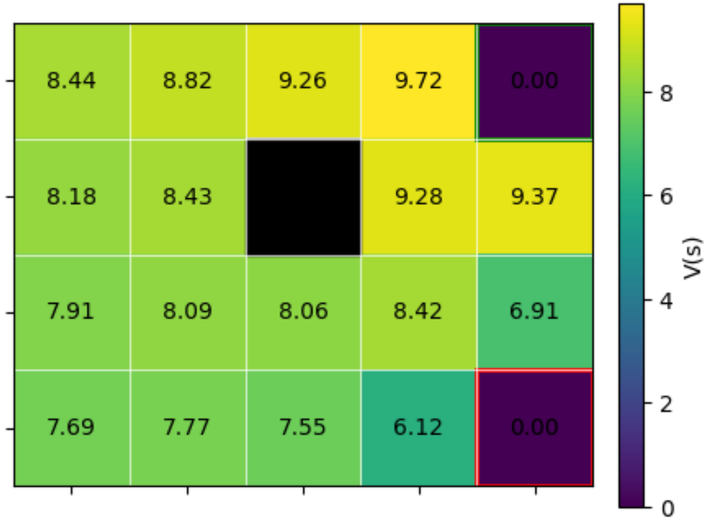


Policy iteration

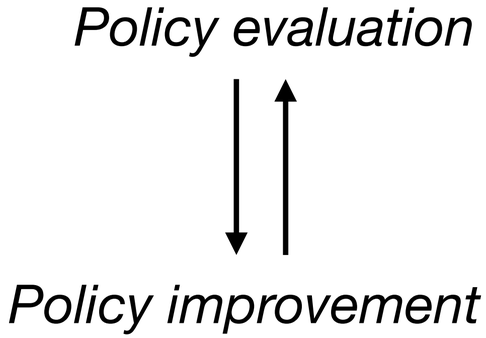
$\pi^*(s)$



$V^*(s)$



Policy iteration:



Wait for complete convergence of value function before proceeding with policy improvement.

Value iteration:

Interleave policy evaluation and improvement into a single iterative process.

More specifically, alternate between policy evaluation and improvement after only a single sweep of the entire state space.

Bellman's optimality equation

Bellman equation $V^{\pi}(s) = \sum_{a \in A} \pi(a|s) \left[R(s,a) + \gamma \sum_{s' \in S} T(s'|s,a) V^{\pi}(s') \right]$

$$V^*(s) = \max_{a \in A} \left[R(s,a) + \gamma \sum_{s' \in S} T(s' \mid s,a) V^*(s') \right]$$

Optimal value at state s

One-step maximization problem.

Optimal value at state s'

Derivation of Bellman's optimality equation

Recall Bellman's equation:

$$V^*(s) = \max_{a \in A} \left[R(s,a) + \gamma \sum_{s' \in S} T(s' \mid s,a) V^*(s') \right]$$

Derivation of Bellman's optimality equation

Recall Bellman's equation:

$$V^*(s) = \max_{a \in \mathcal{A}} \left[R(s,a) + \gamma \sum_{s' \in S} T(s' \mid s, a) V^*(s') \right]$$

And the definition of the optimal value function:

$$V^*(s) = \sup_\pi V^\pi(s)$$

$$V^\pi(s) = \sum_{a \in \mathcal{A}} \pi(a|s) \left[R(s,a) + \gamma \sum_{s' \in S} T(s'|s,a) V^\pi(s') \right]$$

Derivation of Bellman's optimality equation

Recall Bellman's equation:

$$V^*(s) = \max_{a \in A} \left[R(s,a) + \gamma \sum_{s' \in S} T(s' \mid s, a) V^*(s') \right]$$

And the definition of the optimal value function:

$$V^*(s) = \sup_\pi V^\pi(s)$$

Then, we have

$$V^\pi(s) \leq \sum_{a \in A} \pi(a \mid s) \left[R(s,a) + \gamma \sum_{s' \in S} T(s' \mid s, a) V^*(s') \right]$$

$$\implies \sup_\pi V^\pi(s) \leq \sup_\pi \sum_{a \in A} \pi(a \mid s) \left[R(s,a) + \gamma \sum_{s' \in S} T(s' \mid s, a) V^*(s') \right]$$

$$\implies V^*(s) \leq \max_{a \in A} \left[R(s,a) + \gamma \sum_{s' \in S} T(s' \mid s, a) V^*(s') \right]$$

Derivation of Bellman's optimality equation continued

We established that $$V^*(s) \leq \max_{a \in A} \left[R(s,a) + \gamma \sum_{s' \in S} T(s' \mid s, a) V^*(s') \right]$$

Derivation of Bellman's optimality equation continued

We established that $$V^*(s) \leq \max_{a \in A} \left[R(s,a) + \gamma \sum_{s' \in S} T(s' \mid s, a) V^*(s') \right]$$

And, by definition of $V^*(s)$ we have that for every $a \in A$

$$V^*(s) \geq R(s,a) + \gamma \sum_{s' \in S} T(s' \mid s, a) V^*(s')$$

Consider any policy π_a^*. Take action a immediately, and then follow π^* thereafter.

$$V^{\pi_a^*} = R(s,a) + \gamma \sum_{s' \in S} T(s' \mid s, a) V^*(s')$$

$$V^*(s) = \sup_{\pi} V^{\pi}(s)$$

π_a^* is one such policy considered in this supremum.

$$V^*(s) \geq V^{\pi_a^*}(s) = R(s,a) + \gamma \sum_{s' \in S} T(s' \mid s, a) V^*(s')$$

Derivation of Bellman's optimality equation continued

We established that $$V^*(s) \leq \max_{a \in A} \left[R(s,a) + \gamma \sum_{s' \in S} T(s' \mid s,a) V^*(s') \right]$$

And, by definition of $V^*(s)$ we have that for every $a \in A$

$$V^*(s) \geq R(s,a) + \gamma \sum_{s' \in S} T(s' \mid s,a) V^*(s')$$

Which clearly implies $$V^*(s) \geq \max_{a \in A} \left[R(s,a) + \gamma \sum_{s' \in S} T(s' \mid s,a) V^*(s') \right]$$

Derivation of Bellman's optimality equation continued

We established that $$V^*(s) \leq \max_{a \in A} \left[R(s,a) + \gamma \sum_{s' \in S} T(s' \mid s, a) V^*(s') \right]$$

And, by definition of $V^*(s)$ we have that for every $a \in A$

$$V^*(s) \geq R(s,a) + \gamma \sum_{s' \in S} T(s' \mid s, a) V^*(s')$$

Which clearly implies $$V^*(s) \geq \max_{a \in A} \left[R(s,a) + \gamma \sum_{s' \in S} T(s' \mid s, a) V^*(s') \right]$$

Combining these inequalities, we get arrive at the desired result.

$$\boxed{V^*(s) = \max_{a \in A} \left[R(s,a) + \gamma \sum_{s' \in S} T(s' \mid s, a) V^*(s') \right]}$$

Value iteration

Define *Bellman's optimality operator* $T_* : \mathbb{R}^{|S|} \to \mathbb{R}^{|s|}$ as:

$$(T_* V)(s) = \max_{a \in A} \left[R(s,a) + \gamma \sum_{s' \in S} T(s' \mid s, a) V(s) \right]$$

Value iteration operates by turning Bellman's optimality operator T_* into an iterative procedure.

- Pick some initial guess for $V^*(s)$. e.g., $V_0(s) = 0, \forall s \in S$.
- For k = 0, 1, 2, …
  - $V_{k+1} \leftarrow T_* V_k$

Value iteration

Define *Bellman's optimality operator* $T_* : \mathbb{R}^{|S|} \to \mathbb{R}^{|s|}$ as:

$$(T_* V)(s) = \max_{a \in A} \left[R(s,a) + \gamma \sum_{s' \in S} T(s' \mid s, a) V(s) \right]$$

Value iteration operates by turning Bellman's optimality operator T_* into an iterative procedure.

- Pick some initial guess for $V^*(s)$. e.g., $V_0(s) = 0, \forall s \in S$.
- For k = 0, 1, 2, …
  - $V_{k+1} \leftarrow T_* V_k$

The optimal value function V^* is the unique fixed point $V^* = T_* V^*$.

Value iteration

Define *Bellman's optimality operator* $T_* : \mathbb{R}^{|S|} \to \mathbb{R}^{|s|}$ as:

$$(T_* V)(s) = \max_{a \in A} \left[R(s,a) + \gamma \sum_{s' \in S} T(s' \mid s, a) V(s) \right]$$

Value iteration operates by turning Bellman's optimality operator T_* into an iterative procedure.

- Pick some initial guess for $V^*(s)$. e.g., $V_0(s) = 0, \forall s \in S$.
- For k = 0, 1, 2, …
  - $V_{k+1} \leftarrow T_* V_k$

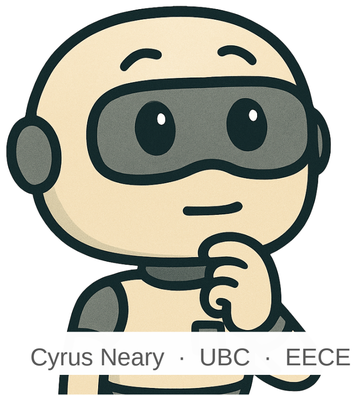
The optimal value function V^* is the unique fixed point $V^* = T_* V^*$.

Given $V^*(s)$, the optimal policy $\pi^*(s)$ may be optained by solving one-step optimization problems at every state:

$$\pi^*(s) \in \text{argmax}_{a \in A} \left[R(s,a) + \gamma \sum_{s' \in S} T(s' \mid s, a) V^*(s) \right]$$

How could we analyze the convergence of value iteration?

- Same as policy iteration
- First step: show that T_π the Bellman optimality operator is a contraction mapping.
- Second step: By Banach fixed point thm, T_* must have a unique fixed point.
- V^* is a fixed point of T_* By definition, so the iterative process converges to V^*

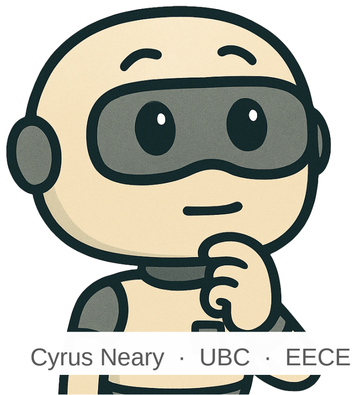


What's the relationship between optimal value functions and policies?

- optimal policy $\Rightarrow$ We can do policy evaluation to find $V^*(s)$.

- optimal value function $\Rightarrow$ We extract the optimal policy by solving a one-step maximization problem

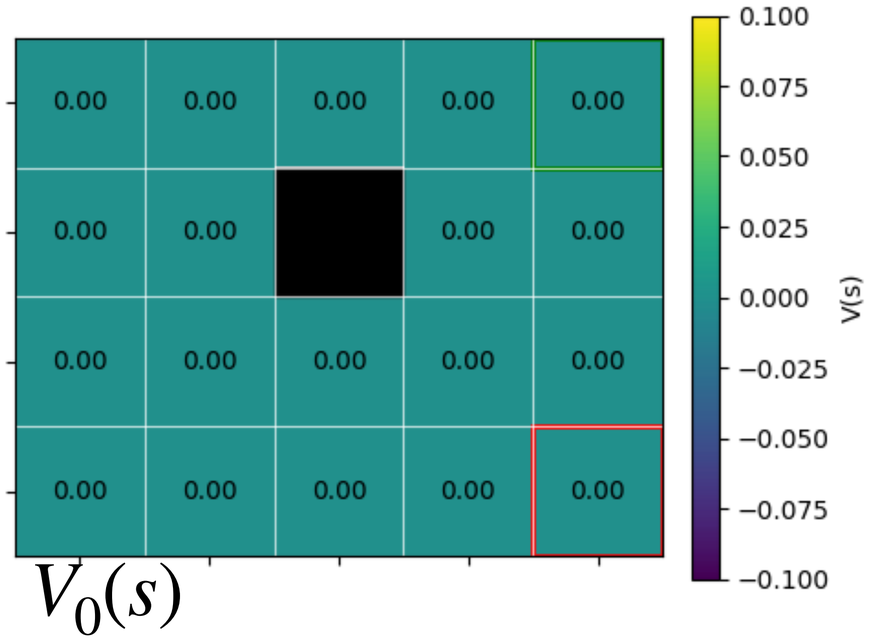
$$\pi^*(s) = \arg\max_{a \in A} \left[R(s,a) + \gamma \sum_{s'} T(s'|s,a) V^*(s') \right]$$



initial objective

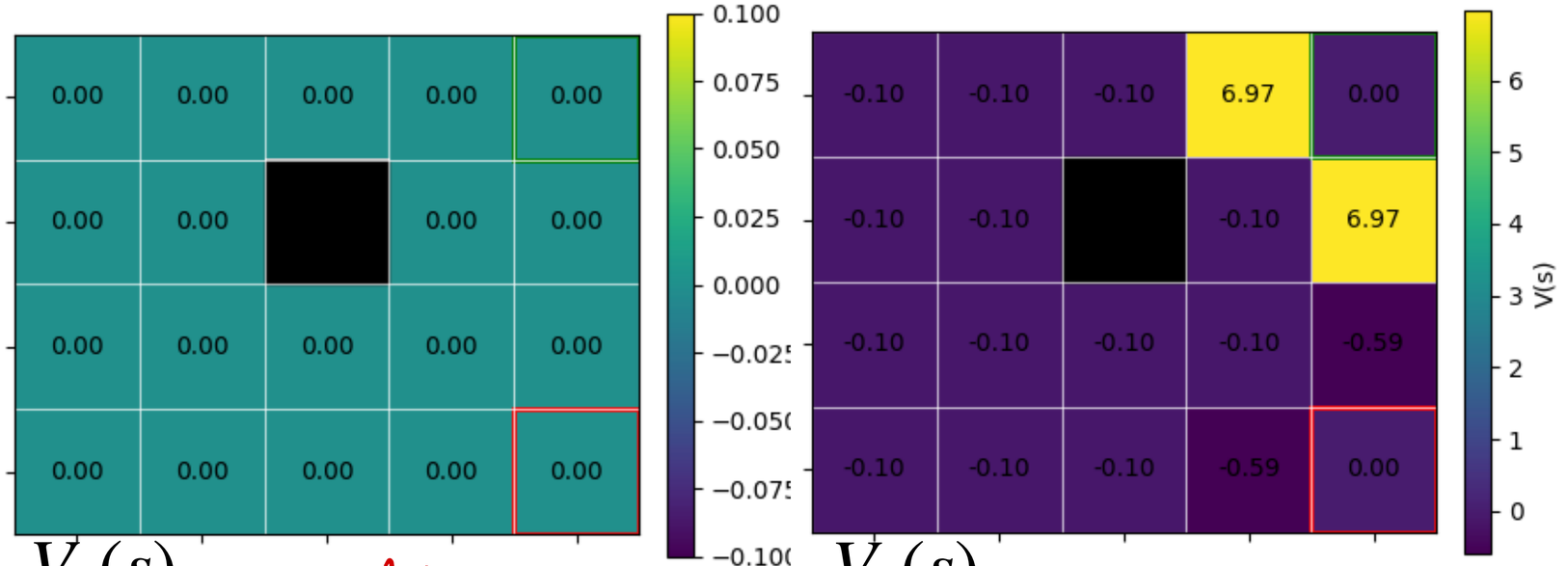
$$\max_{\pi} \mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t r_t \mid \pi, s_0 \right]$$

Gridworld example: value iteration



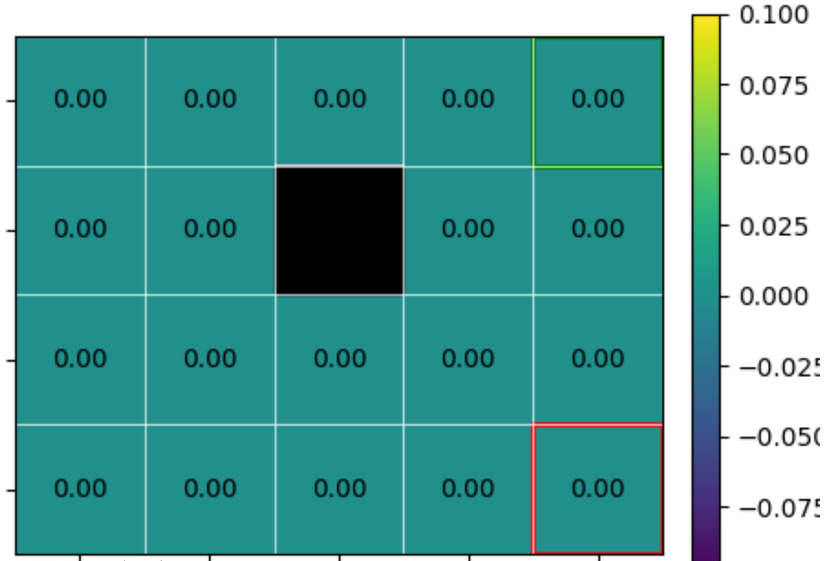
$V_0(s)$

Gridworld example: value iteration



$V_0(s)$ $V_1(s) = T^* V_0(s)$ $V_1(s)$

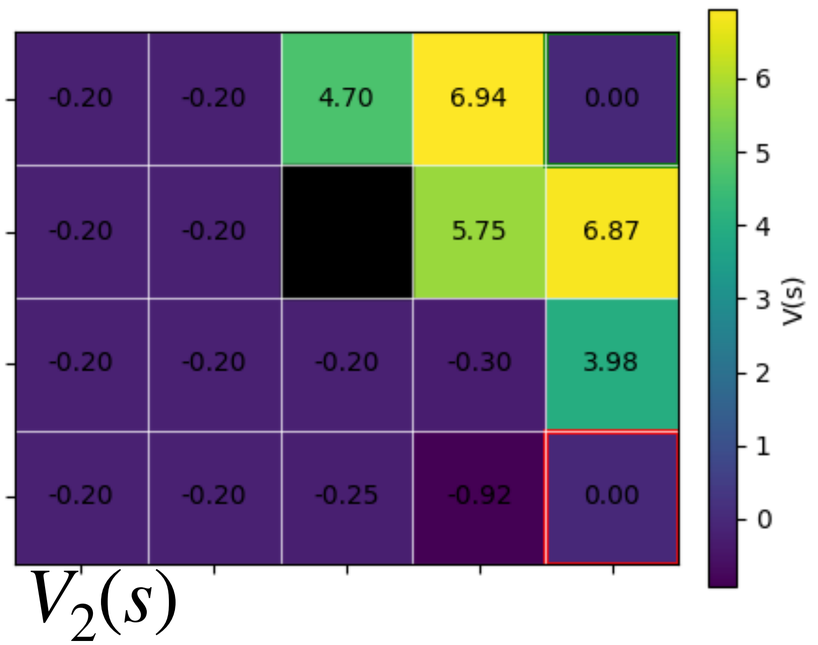
Gridworld example: value iteration



$V_0(s)$

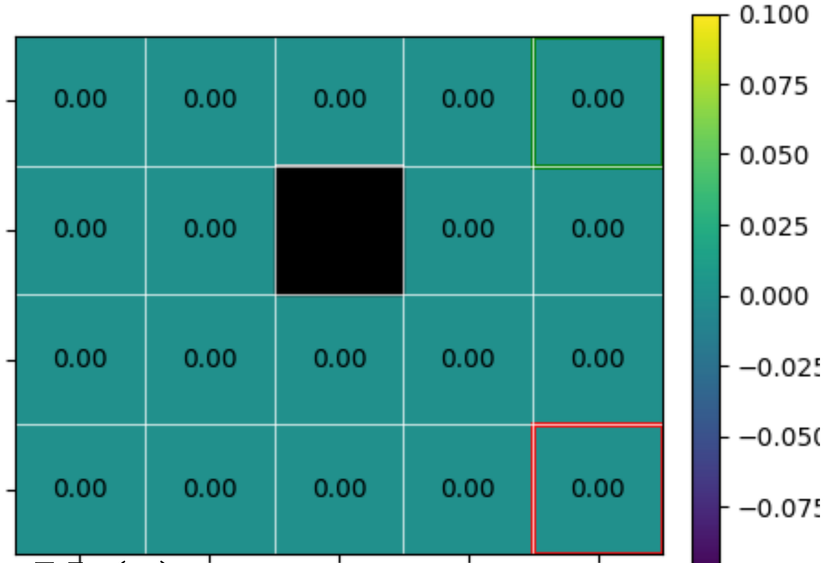


$V_1(s)$



$V_2(s)$

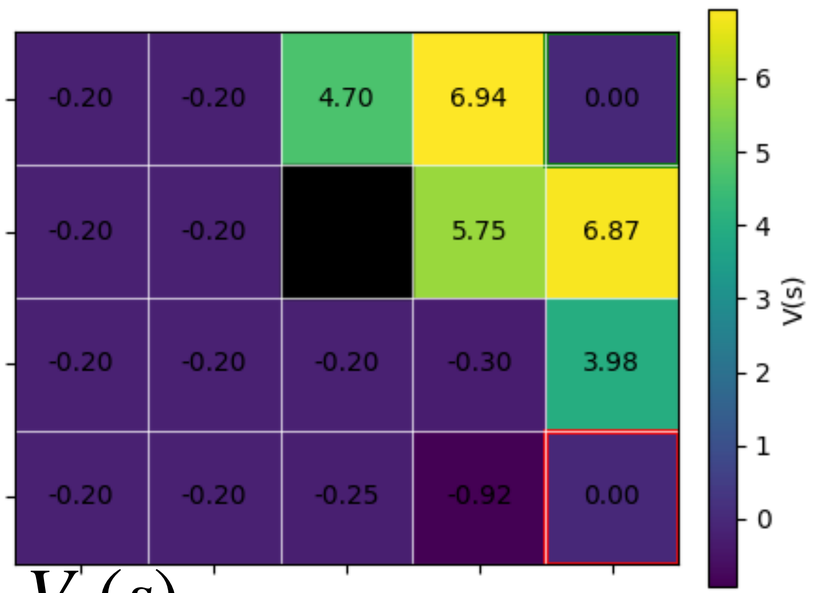
Gridworld example: value iteration



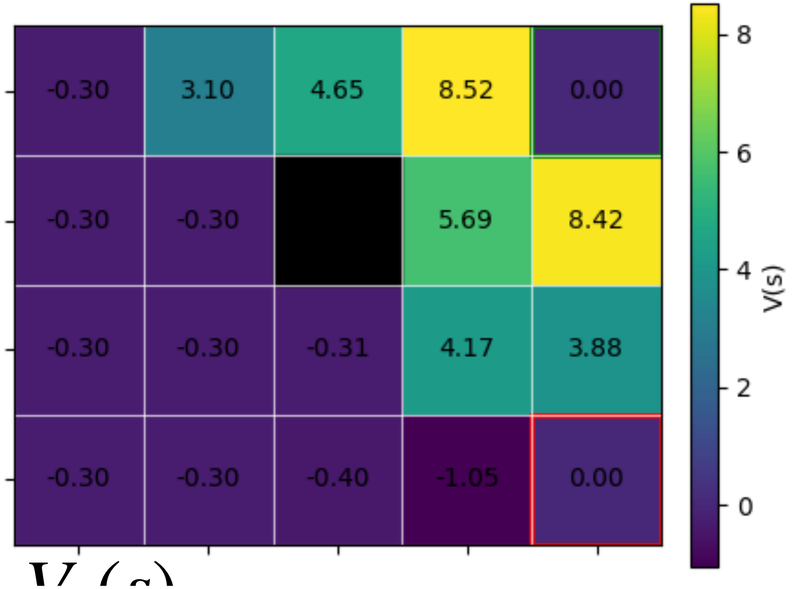
$V_0(s)$



$V_1(s)$

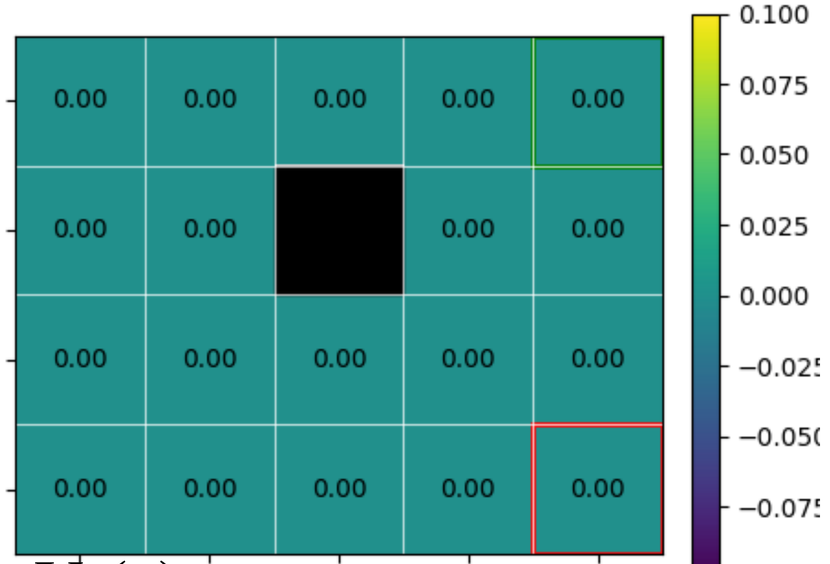


$V_2(s)$



$V_3(s)$

Gridworld example: value iteration



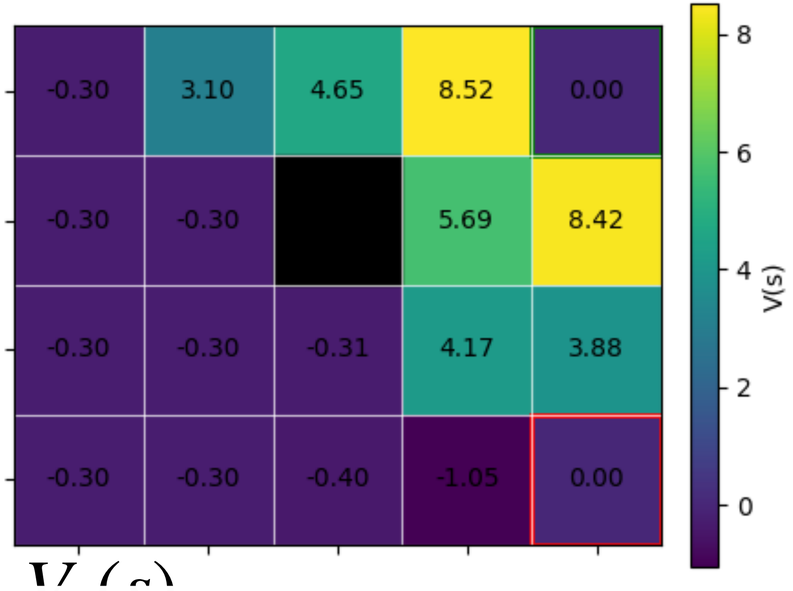
$V_0(s)$



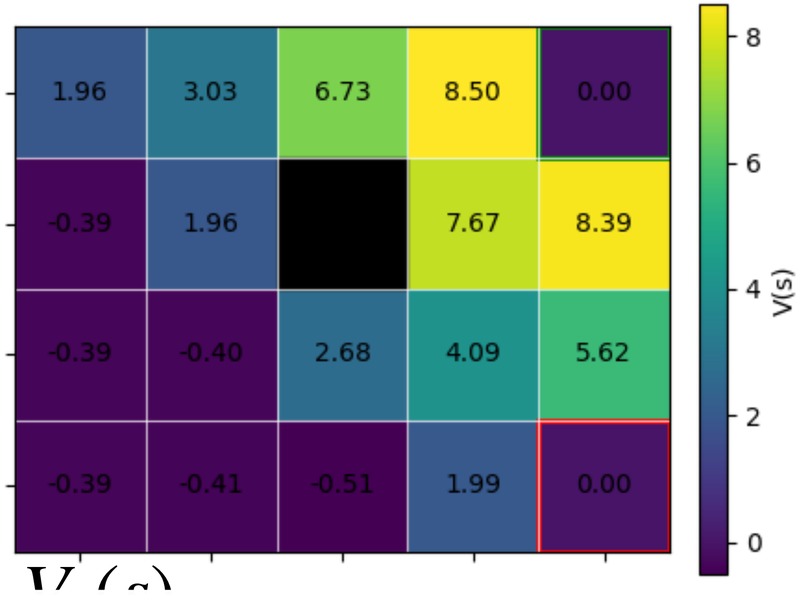
$V_1(s)$



$V_2(s)$

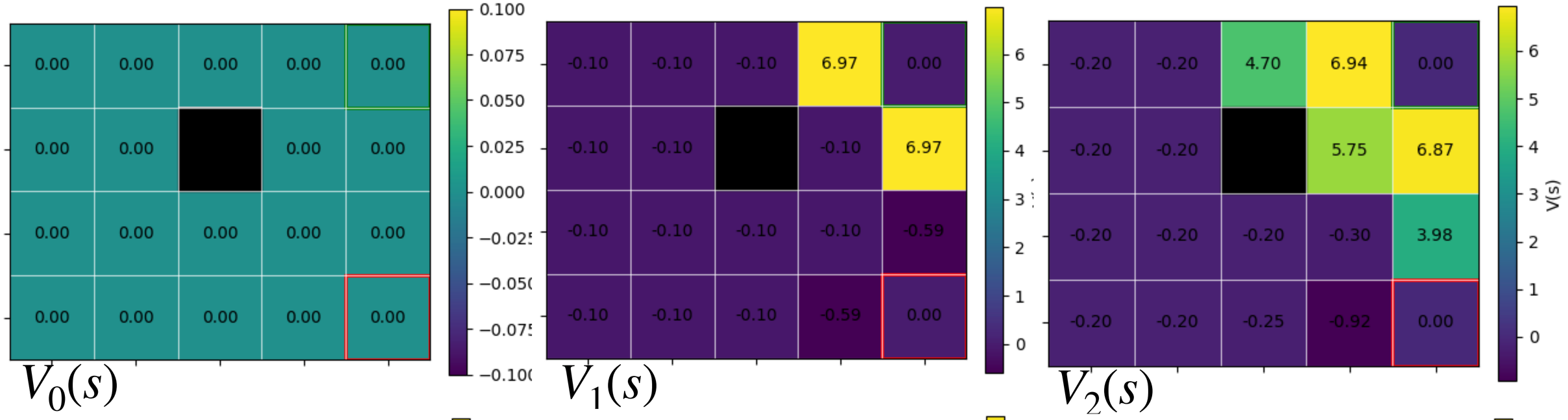


$V_3(s)$



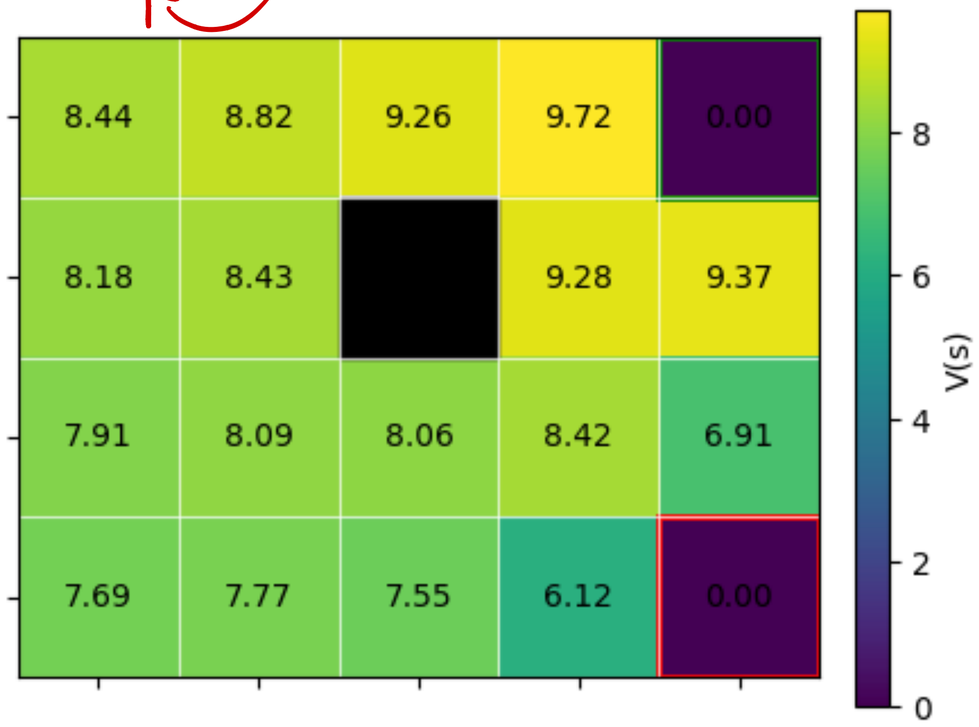
$V_4(s)$

Gridworld example: value iteration

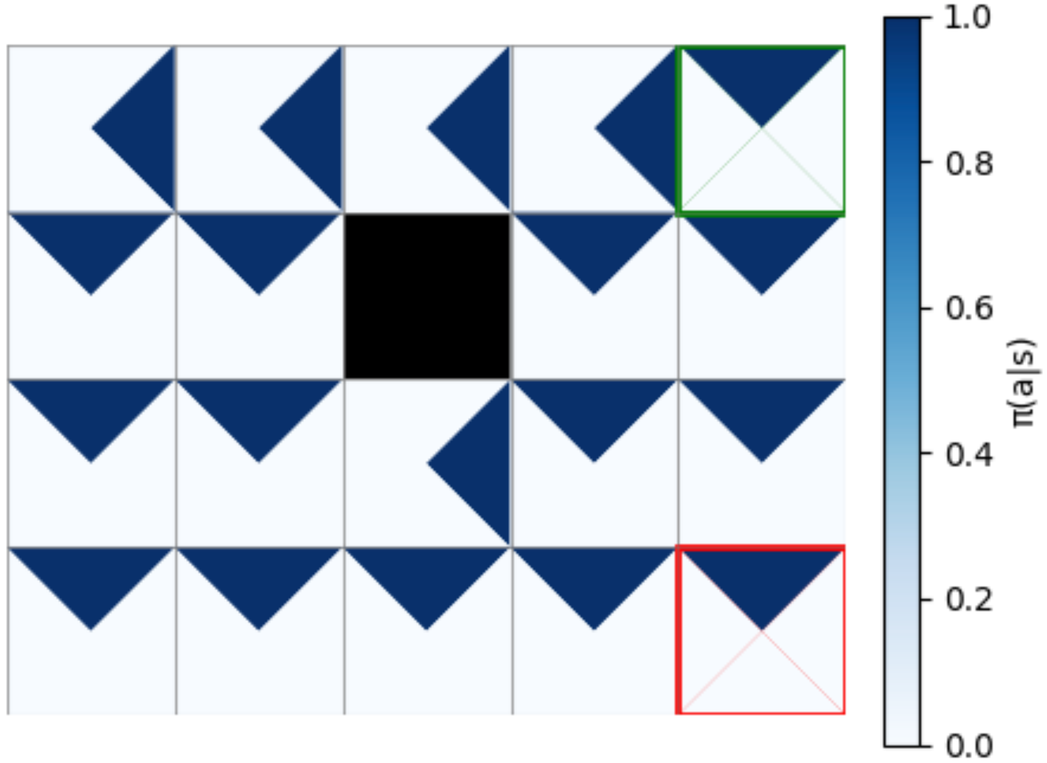


Gridworld example: value iteration

$V_{82}(s)$

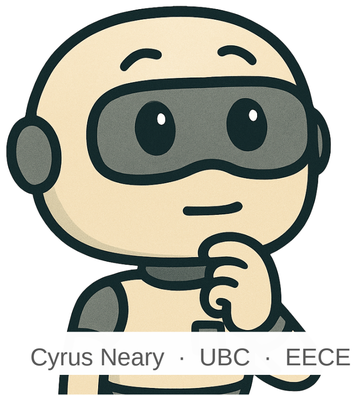


$\pi^*(s)$



What assumptions are required for Bellman's equations to hold?

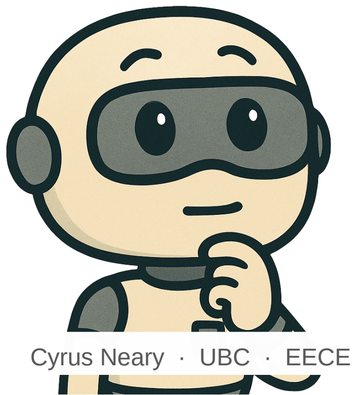
- Time invariance: $T(s'|s,a)$ and $R(s,a)$ are <u>not</u> changing in time.
  - ↳ Quite challenging to apply these algorithms in dynamically changing environments.
- Markov property: $\mathbb{P}(s'|s,a,\text{history}) = T(s'|s,a)$
- Infinite horizons $\sum_{t=0}^{\infty} \gamma^t r_t$
  - ↳ Bellman's equations still hold for finite times, but policies/values will depend on time.
- $R(s,a)$ should be bounded.
- State/action spaces are finite.
- Scalar rewards + values.



What are some practical examples in which the assumptions don't hold?

- Markovian property → Partially observable environments.
  - e.g. Robot that doesn't know it's exact state. Only has camera observations of surroundings.
- Rewards/dynamics that depend on time.
  - Deadline for your project proposal

- No scalar reward function



Summary

Bellman's equation and operator

$$V^{\pi}(s) = \sum_{a \in A} \pi(a \mid s) \left[R(s,a) + \gamma \sum_{s' \in S} T(s' \mid s,a) V^{\pi}(s') \right] \qquad (T_{\pi}V)(s) = \sum_{a \in A} \pi(a \mid s) \left[R(s,a) + \gamma \sum_{s' \in S} T(s' \mid s,a) V(s') \right]$$

Bellman's optimality equation and operator

$$V^{*}(s) = \max_{a \in A} \left[R(s,a) + \gamma \sum_{s' \in S} T(s' \mid s,a) V^{*}(s') \right] \qquad (T_{*}V)(s) = \max_{a \in A} \left[R(s,a) + \gamma \sum_{s' \in S} T(s' \mid s,a) V(s') \right]$$

Policy iteration

Policy evaluation

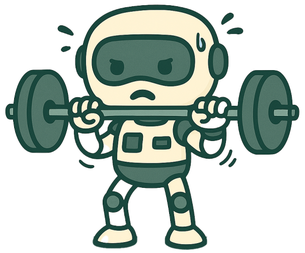
$$V_{\pi} = (I - \gamma P_{\pi})^{-1} R_{\pi}$$

For k = 0, 1, 2, …

$$V_{k+1} \leftarrow T_{\pi} V_k$$

Policy improvement

$$\pi'(s) \in \text{argmax}_{a \in A} \left[R(s,a) + \gamma \sum_{s' \in S} T(s' \mid s,a) V^{\pi}(s') \right]$$

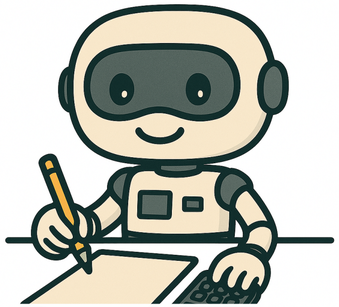


Value iteration

For k = 0, 1, 2, …

$$V_{k+1} \leftarrow T_{*} V_k$$

$$\pi^{*}(s) \in \text{argmax}_{a \in A} \left[R(s,a) + \gamma \sum_{s' \in S} T(s' \mid s,a) V^{*}(s') \right]$$



What is the key difference between policy iteration and value iteration?

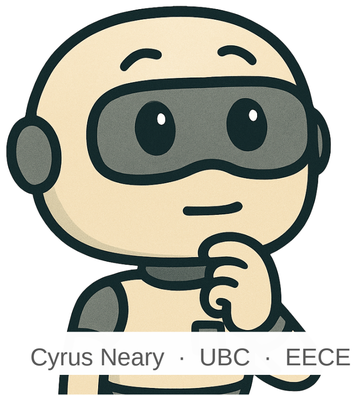
A: Value iteration never uses a Bellman equation.

B: Policy iteration fully evaluates each policy before improving it. Value iteration applies a single Bellman-optimality update per sweep.

C: Value iteration needs a known model T, while policy iteration does not.

D: Policy iteration cannot converge to the optimal policy.

E: I don’t know what’s going on.



Given a policy π and its Q-function Q^{π}, how do we construct an improved policy π', and what does the policy improvement theorem guarantee?

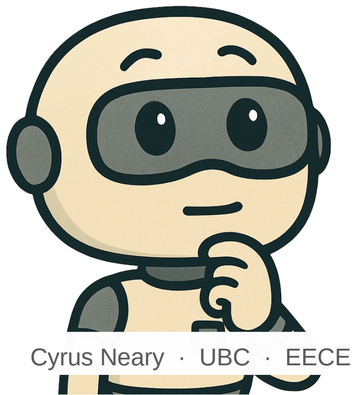
A: $\pi' = \arg\min_a Q^{\pi}(s,a)$, which guarantees $V^{\pi'}(s) \geq V^{\pi}(s), \quad \forall s \in S$

B: $\pi' = \arg\min_a Q^{\pi}(s,a)$, which guarantees $V^{\pi'}(s) \leq V^{\pi}(s), \quad \forall s \in S$

C: π' is chosen randomly and guaranteed to be optimal

D: π' can only be constructed by computing $V^{\pi'}$ first.

E: I don't know what's going on.



Value iteration has converged to $V^*(s)$.

How do you recover an optimal policy π^*?

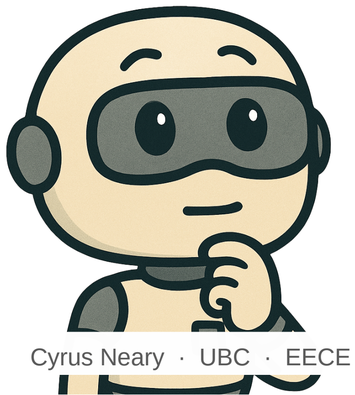
A: Read it off directly as $\pi^*(s) = V^*(s)$

B: It is impossible to recover π^* from V^* without also knowing Q^π for every policy.

C: At each state, pick the action maximizing $R(s,a) + \gamma \sum_{s'} T(s' \mid s,a) V^*(s')$

D: You must restart and run full policy iteration.

E: I don't know what's going on.



Next class

Linear programming solutions to MDPs

&

Connections with optimal control